



红皇后哥德尔机：共同进化的智能体及其评估器

Alex Iacob^{1*}, Andrej Jovanović^{1,3*}, William F. Shen^{1*}, Daniel Burkhardt², Meghdad Kurmanji¹, Nurbek Tastan⁴, Lorenzo Sani^{1,3}, Niccolò Alberto Elia Venanzi², Ambroise Odonnat⁵, Zeyu Cao¹, Bill Marino¹, Xinchu Qiu¹, Nicholas D. Lane^{1,3}

¹University of Cambridge ²NVIDIA ³Flower Labs ⁴MBZUAI ⁵Inria *Equal contribution.

Abstract

自改进智能体在智能体编码基准上已达到最先进水平，且近期已扩展至通用领域。然而，其搜索方法通常假设评估标准是静态的：即一个固定的验证器、基准或标注数据集，在智能体改进过程中始终有效。这忽略了进化的一个核心特征：物种并非针对静态环境进行优化，而是随着环境的变化而适应。本文旨在将这一原理引入递归自改进，使评估成为改进循环的一部分，并开放搜索以纳入不断演化的评估器、对抗性目标以及可能超越静态基准的动态效用。我们提出红皇后哥德尔机（RQGM^[1]），这是一个在非平稳效用下进行递归自改进的进化框架。我们通过受控的效用演化实现这一点：搜索被组织为多个轮次，每个轮次内评估标准固定，而效用可在轮次边界更新，从而在每个轮次内保证自改进，同时目标跨轮次演化。该机制通过共同演化效用本身来实例化，使用与所引导的任务智能体一同改进的学习型评估器。本预印本提供了初步的实证研究。我们首先表明，即使在可验证的编码任务上，RQGM通过添加互补的“智能体作为评审”的代码审查信号，将测试通过率提升至超越先前最先进水平。由于评审器仅被查询一次，而标准编码智能体需要多轮评估，该信号成本更低，且RQGM使用的令牌数减少 **1.35×** – **1.72×**。随后，我们转向科学论文写作与评审，以及奥林匹克级别的证明写作与评分，在这些任务中RQGM的性能超越了先前的自改进智能体：共同演化的作者在多样化的“智能体作为评审”小组下，接受率提高 **1.78×** – **1.86×**，而共同演化的评分者达到高出 **9%** 的真实准确率。由于RQGM可以跨轮次修改搜索目标，它还能对搜索过程本身进行正则化。在论文评审中，最强的基线评审器对人工智能生成论文的过度接受率高达人类的 **1.91×** 倍。我们通过引入对抗性目标来纠正这一问题，该目标能够发现对人工智能和人类工作同样严格的评审器。我们计划在未来的更新中探索更长的搜索视野。

Keywords: self-improving agents, learned evaluation, multi-agent systems, automated scientific discovery, controlled utility evolution, co-evolutionary search



UNIVERSITY OF
CAMBRIDGE



1 引言

自我改进智能体旨在将编码和推理中的局部改进转化为递归循环，使每个更强的智能体能够产生自身的更优变体。达尔文与赫胥黎哥德尔机（Darwin and Huxley Gödel Machines）等搜索方法 [2, 3] 通过编辑自身代码并保留提升外部效用信号的变体，近期在智能体编码任务上树立了开源领域的最先进水平。随后，超智能体（HyperAgents）[4] 将这种自我改进从编码扩展至任意领域的任务智能体。尽管取得了这些进展，当前的搜索方法仍依赖于改进循环之外固定的静态评估，这与生物进化不同——在生物进化中，每个物种都会适应同样在进化的竞争者。这种依赖性还在三种场景下限制了自我改进：当目标任务缺乏直接基准时，例如论文写作和证明写作没有基准，而论文评审和证明评分则有 [5 – 7]；当评估缓慢或信息量弱时 [8 – 10]；以及当静态基准饱和或随着智能体改进而容易受到奖励黑客行为影响时 [11, 12]。

解决这些问题是将自我改进扩展到科学研究与写作等开放式场景的前提。因此，我们提出红皇后哥德尔机（Red Queen Gödel Machine, RQGM），这是一个将评估视为搜索过程一部分的进化框架。我们通过共同进化的学习型评估器来研究这一点，这些评估器与它们所引导的任务智能体一同改进，定义了

¹ 以范·瓦伦（Van Valen）的红皇后假说 [1] 命名：物种通过适应来维持相对于共同进化竞争者的适应度。

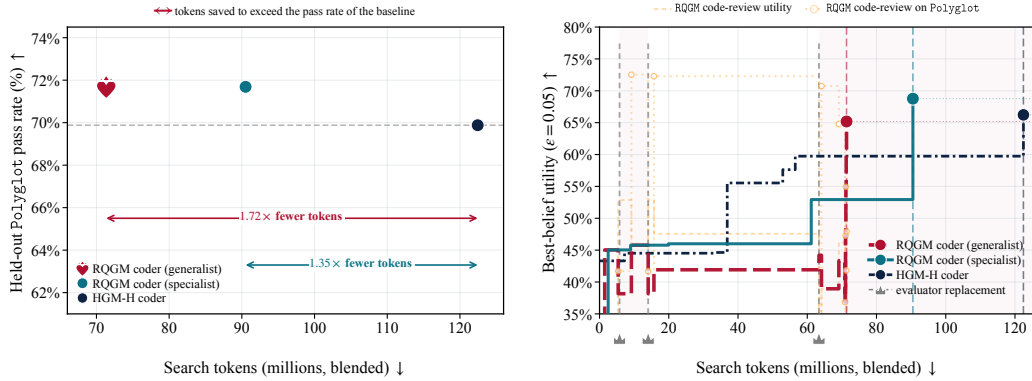


图 1：RQGM 通过添加一个廉价的进化代码评审器，在 **Polyglot** 上以 $1.35\times - 1.72\times$ 更少的搜索令牌超越了先前的 SOTA **HGM-H**。左图：保留集通过率与搜索成本的关系；箭头表示 RQGM 为超越基线通过率所节省的令牌数，心形标记表示该次运行中的最佳智能体。右图：搜索过程中的最佳信念效用。在每次评估器替换（带皇冠的虚线规则）时，效用会下降，因为选择性擦除会丢弃由被替换评审器评分的记录，然后在其替代者下重新攀升；阴影带标记训练轮次。完整细节见第 4 节和第 4.1 节。

搜索效用。这样的评估器可以在没有基准时提供搜索信号，添加基准无法捕捉的标准（如代码可维护性），或提供更廉价的评估代理。

实现这种共同进化的核心机制是受控效用演化，它将搜索划分为多个进化轮次。在一个轮次内，一个评估器被冻结，并为每个任务智能体评分，提供稳定的效用信号；效用只能在轮次边界处改变。同时，搜索在与任务智能体共享的代码库中共同进化挑战者评估器，并根据留出的真实数据集对它们进行评分。在轮次边界处，只有当挑战者在真实数据上统计上优于现有评估器时，它才会取代冻结的评估器；然后，RQGM 应用选择性擦除，仅丢弃依赖于被替换评估器的效用记录。因此，由于每个轮次都是一个固定标准的搜索问题，先前的自我改进保证直接适用（第 3 节和附录 F），而搜索目标可以跨轮次演化，例如通过提升更强的评估器或添加对抗样本正则化项。

我们按照张等人的方法，对三个领域的递归质量生成模型（RQGM）进行了初步实证研究 [4]：编程 [13]；论文写作与审稿 [14, 15]；以及证明写作与评分 [4, 7]。在可验证的编程任务上，通过增加一个作为评判者的智能体代码审阅器，RQGM 的留出测试通过率达到 **71.7%**，而此前最先进水平为 69.9%（图 1）。其质量信号与测试执行互补，提高了通过率，同时与多轮编程智能体执行相比所需令牌更少。这体现了协同进化更广泛的优势：共享扩展可以同时提升多种效用（见图 10）。随后我们转向科学论文写作与审稿，以及奥林匹克级别的证明写作与评分，在这些任务上 RQGM 超越了此前的自我改进智能体：协同进化的写作者将其论文的接受率从此前最先进水平的 21.8% 提高到 **40.5%**（表 1），而协同进化的评分者以比此前最先进水平低 **3x** 的搜索成本超越了静态基线（图 5）。

我们进一步观察到，随着评估者在各个训练轮次中不断增强，它们可能对任务智能体施加类似课程学习的效果，每次转变都会重新排序档案（图 3 和图 4）。协同进化还可以规范评估者：最强的基线审稿人相对于人类论文过度接受人工智能生成的论文 [16]，但 RQGM 的进化轮次让我们能够先收集被冻结评估者接受的人工智能生成样本，然后在后续轮次中将其作为对抗样本重放，从而产生一个对机器和人类工作同样严格的审稿人（图 6）。

我们认为 RQGM 指向一类自我改进的协同进化系统，其中智能体和评估者在没有进一步人类干预的情况下递归地相互引导，以达到超越静态评估的能力。如果成功，这可能是迈向能力更强的 AI 系统的一个有意义的步骤，代价是与静态评估标准相比放宽了收敛保证。

Contributions:

1. 非平稳效用下的递归自我改进。RQGM 将效用信号视为搜索过程的一部分，从而将自我改进智能体扩展到固定外部评估之外。这使得搜索目标得以进化，为自我改进开辟了学习到的非平稳效用（第 3 节）。
2. 受控效用演化。我们通过受控效用演化使非平稳效用与先前的自我改进保证兼容：搜索在固定评估器的训练轮次中运行，目标仅在轮次边界处改变。挑战者评估器只有在提升 ϵ -最佳信念分数（基于真实标签）时才被提升，其效用超过该下界的概率为 $1 - \epsilon$ （第 3 节与附录 F）。
3. **Practical Benefits of Co-Evolved Learned Evaluators and Search Interventions.** Empirically, co-演化增加了一个低成本的已学习代码审查信号，即使在存在验证器的情况下也能提高搜索效率（第 5.1 节），在无法获得直接基准时实现生成器搜索（第 5.2 节），并通过对抗性目标实现对过于宽松评估器的去偏（第 5.4 节）。

2 相关工作

自我改进 人工智能。哥德尔机 [17, 18] 是一种理论构造，只要能够证明某项改变有益，就通过自我修改来改进自身，而元学习则直接优化学习动态 [19, 20]。近期一系列系统通过用对观测效用的档案搜索取代其不可行的证明搜索，使哥德尔机变得可经验化：达尔文-哥德尔机 (DGM) [2] 按基准效用对自我修改的编码智能体进行排序，赫胥黎-哥德尔机 (HGM) [3] 按节点整个后代分支的效用而非其自身效用来评分，而超智能体 [4] 通过为每个档案节点配备元智能体/任务智能体对，将自我修改扩展到编码之外。相关工作还包括自我修改完整代码仓库 [21 – 23]、优化提示 [24] 或推理轨迹 [25]，或针对特定领域进行自我调优 [26]，其中一些仅限于爬山式改进而非开放式探索 [27]。它们的共同点是在整个运行期间保持一个固定的、位于循环之外的评估标准，这使它们容易受到奖励黑客行为 [28] 以及基于模型的评审的位置、冗长和自我偏好偏差 [16, 29, 30] 的影响。随时间更新标准是一种应对方式，但基准创建成本高昂 [11, 12]，且任何改变都会迫使搜索重新开始。我们则让评估器与其评分的智能体共同演化，将自我改进扩展到完全没有直接基准的领域。

大模型作为评审与自动化科学发现。本文所学习的评估器建立在将大语言模型用作评判者的基础上 [29, 31]，并以与人类偏好的一致性为基准进行评测 [15]。自动化发现流程大规模应用此类评判器，但与上述自改进系统类似，它们在每次运行中保持评估固定不变 [6, 14, 32 – 35]；本文则让评判器本身随着搜索的进行而不断改进。

多智能体协同进化。针对移动目标的协同进化是一个经典思想：适应度共享 [36] 和自修改策略 [37] 使种群协同适应，自我博弈则让智能体与自身更强的版本对抗 [38, 39]。近期的大语言模型智能体方法将此思想应用于交互智能体，但保持评估固定 [40, 41]，而人机协同改进则根据反馈调整信号 [42]。RQGM 自动协同进化评估器，因此目标就是学习到的效用本身。

开放式。开放式搜索旨在持续生成新颖工件 [43, 44]，通常通过维护质量-多样性档案 [45, 46] 来实现，并且越来越多地由基础模型 [14, 34, 47] 驱动，以追求封闭自指环这一长期目标 [48]。这些方法仍然针对静态目标进行评估；协同进化目标正是 RQGM 所补充的缺失环节。

3 RQGM：协同进化智能体及其评估器

本文提出红皇后哥德尔机 (Red Queen Gödel Machine)，简称 RQGM，这是一个递归自改进框架，其中学习到的评估器与它们所评分的智能体一同改进（图 2）。在先前系统搜索方法的基础上 [2 – 4]，RQGM 进行了四项修改：(i) 每个档案节点是一个多智能体工作区，而非单个智能体；(ii) 评估器本身是学习到的智能体过程；(iii) 驱动搜索的效用函数可在指定的进化轮次发生变化；以及

(iv) 评估器可在这些时间点被替换。本文在第 3.1 节回顾搜索方法，在第 3.2 和 3.3 节介绍多智能体公式化表述，在第 3.4 和 3.5 节介绍协同进化机制。算法 1 展示了完整流程，附录 F 提供了理论。

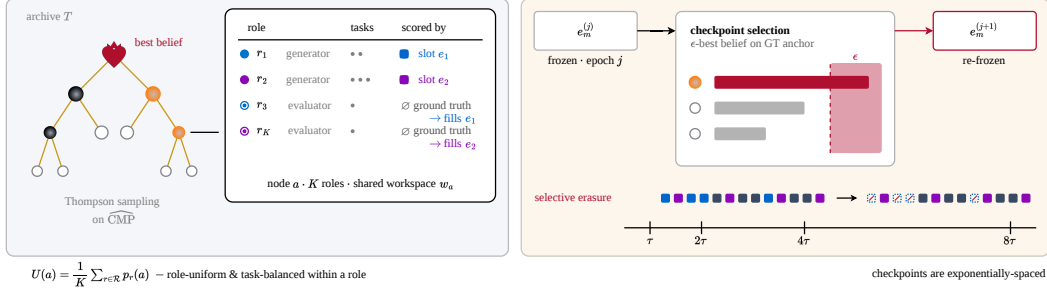


图 2: RQGM 在包含可学习任务智能体和评估器的多智能体工作空间树上进行搜索。 每一步，通过汤普森采样（Thompson Sampling）在分支元生产力上选择一个节点，并由元智能体扩展或评估。评估器根据真实锚点进行评分，任务智能体则根据其轮次局部冻结评估器或针对评估器无关角色的固定基准进行评分。在检查点，每个冻结槽位在真实锚点上将其现任者与挑战者评估器进行比较；然后， ϵ -最佳信念评估器被冻结用于下一轮次，被替换评估器的效用记录被清除（算法 1）。

3.1 预备知识：自我改进作为树搜索

遵循 Wang 等人 [3] 的方法，我们将自我改进形式化为在持续增长的智能体节点档案 $\mathcal{T}_t$ 上的树搜索，初始化为种子节点 $\mathcal{T}_0 = \{a_0\}$ 。在每一步 $t \in \mathbb{N}$ ，搜索要么修改节点 $a \in \mathcal{T}_t$ ，将编辑后的副本添加为 a 的子节点，要么在任务上评估节点，记录二元结果 $o \in \{0, 1\}$ 。这些二元结果被聚合为分数，统计节点在多次评估中累积的成功次数 $S_a \in \mathbb{N}$ 和失败次数 $F_a \in \mathbb{N}$ 。当分配的搜索评估预算 $B \in \mathbb{N}$ 耗尽时，返回得分最高的档案节点。由于节点自身的成功率可能无法很好地预测其后代的表现，我们遵循 HGM[3] 并采用分支元生产力（CMP）作为搜索效用：即节点分支 $C(a)$ 上汇总的成功率，该分支定义为以 a 为根的子树，通过汤普森采样在这些分支级结果上选择节点进行扩展或评估。令 $n_{\text{success}}^C(a), n_{\text{failure}}^C(a) \in \mathbb{N}$ 累积分支 $C(a)$ 中所有节点的成功和失败，我们定义：

$$\text{CMP}(a) = \text{最终智能体}^{\text{CMP}} \frac{n_{\text{success}}^C(a)}{n_{\text{success}}^C(a) + n_{\text{failure}}^C(a)} \in [0, 1] \text{ 最佳信念分数 } BB_\epsilon(a)$$

$$) = I_\epsilon^{-1}(1 + S_a, 1 + F_a) \in [0, 1] \text{ 选择, 其中 } \epsilon \in (0, 1) \text{ 是置信水平, } I_\epsilon^{-1} \text{ 是逆正则化不完全贝塔函数, 即智能体成功 } S_a \text{ 和失败 } F_a \text{ 的贝塔后验的 } \epsilon\text{-分位数, 这是一种保守的效用估计。对于可重复的评估标准, CMP 足以近似哥德尔机 [3]。}$$

3.2 搜索空间

多智能体架构。 为了共同进化智能体与评估器，我们将每个档案节点 a 构建为单一智能体，但作为一个共享的、可进化的、由 $K \in \mathbb{N}$ 个智能体角色 $\mathcal{R} = \{r_1, \dots, r_K\}$ 填充的工作空间，这些角色由任务智能体和评估器组成。每个角色 $r \in \mathcal{R}$ 拥有自己的任务池 $\mathcal{D}_r$ ，即其被评分的有限任务集合，每个任务 $d \in \mathcal{D}_r$ 贡献一个效用信号。例如，论文审稿人角色可能承担两项任务：审阅人类撰写的论文和智能体生成的论文。由于每个任务属于单一角色，任务唯一标识其角色；遵循 HyperAgents，因此在角色无歧义时，我们将角色的智能体称为任务智能体。每个节点分配一个元智能体，负责监督工作空间并可在搜索过程中修改它 [4]。由于元智能体和角色智能体的代码均可编辑，若元智能体认为有用，角色可以复用彼此的代码。仅评分和编排框架是固定的。

学习型评估器。 诸如论文写作之类的开放式角色没有真实信号，需要学习型评估器。我们将此类角色称为评估器依赖型，与具有固定基准的评估器独立型角色形成对比。每个评估器依赖型角色通过一个可在搜索过程中替换的评估器槽位进行评分；其轮次索引记录替换次数。由于 $M \leq K$ 个评估

器依赖型槽位独立替换，我们将它们的轮次索引收集为轮次向量 $\mathbf{j} = (j_1, \dots, j_M) \in \mathbb{N}^M$ 。替换槽位的评估器会推进其条目 $\mathbb{N}^M \ni \mathbf{j} \rightarrow \mathbf{j}' \in \mathbb{N}^M$ (第 3.5 节)，因此每个评估器与其角色共同进化，同时保持轮次平稳。

效用定义。 效用通过两次均匀平均自底向上聚合，首先对角色内的任务求平均，然后对角色求平均，因此搜索因优化所有角色而获得同等奖励。² 基础量是 $p_{r,d,j}(a) \in [0, 1]$ ，节点 a 's 在轮次向量 j ；下对任务 $d \in \mathcal{D}_r$ 的期望二元成功率， j 下标使该成功率以及由此构建的所有效用成为轮次局部的。对角色内任务均匀平均得到每角色成功率 $p_{r,j}(a) = \frac{1}{|\mathcal{D}_r|} \sum_{d \in \mathcal{D}_r} p_{r,d,j}(a) \in [0, 1]$ ，再对角色均匀平均得到智能体效用 $U_j(a) = \frac{1}{|\mathcal{R}|} \sum_{r \in \mathcal{R}} p_{r,j}(a) \in [0, 1]$ 。

数据隔离。 为防止记忆并提升泛化能力，RQGM 将产生节点的反馈与选择节点的证据分离。对于与评估器无关的角色，每个节点仅看到训练划分的一个子集，验证与测试划分保持独立。对于与评估器相关的角色，任务智能体生成新的工件，由训练轮次局部评估器评分，训练与搜索使用不同的工件。在两种情况下，每个节点及其祖先的训练结果都会展示给元智能体以指导自我修改，但绝不用影响搜索效用。节点选择仅由验证划分上的评估驱动，从而防止过拟合。最终性能在独立于搜索所用验证数据的留出测试划分上报告。

3.3 三级采样层级

遵循 Wang 等人 [3] 的方法，调度器将扩展（添加新节点）与评估（对现有节点评分）交错进行，同时平衡探索与利用以寻找良好解。在每一步 t ，给定已执行的评估次数 $N_t \in \mathbb{N}$ 、扩展指数 $\alpha \in (0, 1)$ 以及档案大小 $|\mathcal{T}_t|$ ，a，UCB-Air 门 [49] 在 $N_t^\alpha \geq |\mathcal{T}_t|$ 时添加新节点，否则评估现有节点。

因此，档案的大小为 $|\mathcal{T}_t| = O(N_t^\alpha)$ 。

调度器首先通过汤普森采样在 CMP 上选择一个节点 $a^\star$ ：扩展操作将 $a^\star$ 交给其元智能体以编辑成新的子节点，而评估操作则下降两级，先角色后任务。设 $\mathcal{R}_{\text{elig}} \subseteq \mathcal{R}$ 为节点 $a^\star, \mathcal{D}$ 处符合条件的

角色， $\mathcal{D}_{r,\text{elig}} \subseteq \mathcal{D}_r$ 为角色 r 的符合条件的任务， $n_r(a^\star), n_d(a^\star) \in \mathbb{N}$ 为节点 $a^\star$ 处分配给角色 r 和任务 d 的评估次数。除非数据集在采样下耗尽，否则任务均符合条件；当采用有放回采样时，每个任务始终符合条件（ $\mathcal{D}_{r,\text{elig}} = \mathcal{D}_r$ ），且只要存在符合条件的任务，角色即符合条件。在每一级，调度器选择评估次数最少的符合条件的选项：角色 $r^\star = \arg \min_{r \in \mathcal{R}_{\text{elig}}} n_r(a^\star)$ 以 $a^\star$ 为条件，然后任务 $d^\star = \arg \min_{d \in \mathcal{D}_{r^\star,\text{elig}}} n_d(a^\star)$ 以 $(a^\star, r^\star)$ 为条件。每次评估产生一个二元结果，在节点级累积用于汤普森采样；角色级和任务级计数器仅用于负载均衡。命题 1 表明，对于固定的评估器训练轮次和节点，跨角色和任务的均衡采样使得合并的 Beta 累加器的后验均值几乎必然收敛到角色-任务均衡效用 $U_j(a)$ 。

3.4 协同演化评估

我们假设学习到的评估器在训练轮次内是局部平稳的（假设 2）：评估器、工件生成协议和二元评分规则在整个训练轮次内保持不变。我们将效用转换（定义 F.2）定义为一种过程，该过程替换槽位的评估器并执行选择性擦除，仅移除与被替换槽位相关的效用历史，同时保留所有无关信息。当多个转换同时触发时，选择性擦除也是顺序无关的（备注 1）。

训练轮次内局部有效性。 如果依赖评估器的标准在每个训练轮次内固定，并且在转换时擦除被替换评估器所失效的记录，那么每个训练轮次都会诱导出一个固定的二元结果搜索问题（命题 3）。因此，RQGM 可以直接在训练轮次内使用 HGM 搜索。

3.5 受控效用演化

虽然固定评估器为每个训练轮次提供了平稳的效用信号，但 RQGM 通过在训练轮次边界上依据原则性标准替换评估器来逐步改进它们。更广泛地说，评估器替换是效用转换（定义 F.2）的一个例子。由于我们的保证仅要求训练轮次内固定的效用标准和边界处对依赖记录的擦除，其他非平稳效用（如时变基准或对抗性目标）也符合相同的结构。

基于真实最佳信念的评估器替换。 在每个训练轮次边界，每个槽位将其现任评估器与挑战者在真实锚点上进行比较：一个固定的、留出的客观数据集或

² 我们默认采用均匀加权而非样本加权平均，因为某些任务可能产生无界评估。

该角色的人类偏好标签。由于该锚点独立于评估器，与其一致可提供跨训练轮次一致的替代标准。候选者按用于智能体选择的同一 ϵ 最优信念分数 (BB_ϵ) 排序 (第 3.1 节)，最大锚点 BB_ϵ 被冻结为下一训练轮次的评估器，平局时偏向现任者以避免不必要的擦除。命题 5 将多训练轮次过程确立为一系列固定标准搜索。

锚点下界。 晋升的评估器最大化 BB_ϵ ，这是其锚点效用以概率 $1 - \epsilon$ 超过的下界，因此替代者很可能优于现任者 (命题 4 与备注 3)。

摊销效用转换成本。 当评估器被替换时，由被替换评估器评分的记录将被擦除，受影响的智能体必须在新评估器下重新排序。立即重新排序整个档案会浪费预算在搜索可能不会再次访问的节点上。相反，RQGM 仅在后续评估返回旧节点时才重新评分。三级采样层次 (第 3.3 节) 在搜索进行时自然优先考虑评估不足的节点、角色和任务。重用缓存的智能体输出也使我们无需额外调用任务智能体即可重新评估节点。

擦除保持训练轮次局部平稳性：保留过时效用或将其重新缩放到新评估器会混合来自不同效用函数的证据，并违反我们保证所需的条件。为控制重新评估成本，我们引入检查点计划。采用指数间隔检查点，暴露于擦除的记录数量仅随评估预算线性增长，这也使后续更准确的评估器能够更长时间地塑造搜索。

有界恢复。 对于比率为 $\rho > 1$ 的指数间隔检查点，在 B 次评估预算内暴露于擦除或重新评估的先前槽位相关记录的累计数量为 $O(B)$ 。这降低了每次评估后允许效用转换的 $O(B^2)$ 成本 (命题 6 与备注 7)。

理论保证总结： 命题 3、命题 6 与备注 3 共同表明，异质梯度方法 (HGM) 的自我改进保证通过受控效用演化，在逐训练轮次的基础上延续至协同演化场景。这依赖于以下三个结果：第一个结果确保每个训练轮次内的保证，第二个结果实现跨训练轮次的改进传递，第三个结果界定实现这一过程的代价。

1. 训练轮次局部有效性。每个训练轮次本身是一个固定准则的搜索问题 (命题 3)，因此异质梯度方法的逐训练轮次收敛保证可直接应用于每个训练轮次内部。
2. 锚点引导改进。评估器仅在提升固定真实锚点上的 ϵ -最佳信念分数时才被提升 (备注 3)，因此评估器不仅在训练轮次内改进，而且跨训练轮次改进。
3. 摊销转换。评估器替换的簿记成本与搜索预算呈线性关系 (命题 6)，因此协同演化不会在标准档案搜索之外引入渐近开销。

非正式地说，由于训练轮次局部平稳性，收敛保证对随机量子几何方法 (RQGM) 逐训练轮次成立，而受控效用演化鼓励跨训练轮次边界的改进。

4 实验设计

我们的实验设计回答了以下关于随机量子几何方法的研究问题：

研究问题一 在存在真实验证但代价高昂的领域，真实梯度匹配能否带来益处？**研究问题二：** 在缺乏客观评估的领域，学习式评估能否改进智能体？**研究问题三：** 评估器替换能否随时间引导搜索找到更优的任务智能体？**研究问题四：** 协同进化能否改进评估器本身？

领域与真实锚点。 每个领域将一个生成器角色与一个学习式评估器角色及一个真实锚点配对 (图 8)。论文写作与论文评审配对，锚定于自动论文评审系统的接受/拒绝决策 [15]；我们构建了一个匹配的自动论文评审系统标题与摘要写作数据集，以对齐各角色的训练、验证和测试分布。证明写作与证明评分配对，锚定于国际数学奥林匹克评分基准的人工评分 [7]，其中证明仅当该训练轮次冻结的评分器给予满分 (7 分中的 7 分) 时才被接受。编码使用两个锚点：编码智能体锚定于可执行的多语言测试 [13]，而协同进化的代码评审器锚定于代码评审自动评估数据集 [50] (一个已接受/已拒绝拉取请求的数据集)，并在生成时对每个多语言补丁进行评分。跨领域，每个角色智能体均从最小模板初始化 (附录 C.5)。软件工程基准 [51] 运行时间过长，因此未纳入本版本工作。

专家与通才。与先前方法不同，真实梯度匹配同时针对多个效用函数优化智能体。这引出一个问题：在选择最佳信念智能体时应考虑哪些效用。我们将专家定义为在某一目标效用上具有最高最佳信念的智能体，将通才定义为在所有活跃效用上具有最高平均最佳信念的智能体。同时报告两者可显示目标任务性能是来自任务特定优化还是联合优化。

基线。每个领域使用一个学习式基线，即分层梯度匹配-分层，它将超智能体 [4] 的分布式梯度匹配搜索算法替换为样本效率更高的分层梯度匹配，同时保持评估器冻结。在每个领域中，分层梯度匹配-分层使用来自先前工作的固定外部评估器。我们使用鱼子酱提示评审器 [14] 进行论文写作，使用证明自动评分器 [7] 进行证明评分，并使用国际数学奥林匹克 25 证明器 [52]。我们还评估了张等人 [4] 发表的用于论文评审和评分的最佳超智能体（分布式梯度匹配-分层）补丁，作为分布式梯度匹配级别性能的代理，尽管它们是在不同的基础模型上训练的。张等人 [4] 未发布最佳多语言智能体。

有原则的搜索干预。协同演化框架允许在训练轮次边界对搜索目标进行有原则的修改，同时保持训练轮次局部的收敛保证。在论文领域，我们利用这一点来纠正大语言模型的自偏好偏差（第 2 节）：在第一次替换之后，被替换审稿人接受的论文形成一个对抗池，随后的训练轮次额外奖励拒绝这些作者生成的论文，同时在人类 APReS 数据上保持准确率。评估器替换本身仍锚定于 APReS（附录 C）。隔离 RQGM 各部分的额外消融实验报告于附录 D.2。我们选择 GPT-5.5（低）用于正文实验，因为它提供了理想的成本-智能权衡（附录 C）。

成本与报告。我们将搜索过程中达到最高 ϵ -最佳信念分数的智能体称为最佳信念智能体。该分数只有在基于固定真值锚点时才能跨训练轮次比较，因此全局最佳信念赢家仅存在于锚定角色：学习到的评估器和可验证的编码智能体。论文作者和证明证明者没有锚点，只承认训练轮次局部赢家，每个赢家都是在其训练轮次冻结评估器下的最佳智能体；我们按训练轮次选择它们，并在事后用固定的外部评判器评分（表 1 和表 2）。搜索预算在各运行之间匹配，因此成本差异反映搜索动态而非分配不均，我们沿三个轴比较运行：匹配计算量下的最佳智能体、达到匹配质量的成本以及总体最佳。

由于当评估成本变化时，相等的评估预算可能掩盖巨大的计算差异，我们报告混合令牌：输入加输出，输出按 $5\times$ 输入成本 [53, 54] 加权，同时计入生成和评估调用。遵循 Wang 等人 [3]，最佳信念智能体的成本是它首次达到最高 ϵ -最佳信念分数时消耗的预算，而不是训练轮次或运行结束时。不确定性以 95% 中心贝塔（Jeffreys）区间报告；更多细节见附录 C。

4.1 可视化指南

所有结果图（图 1、图 5 和图 6）共用同一方案，比较多个臂：搜索配置，如 RQGM 通才、专才和 HGM-H 基线。每幅图将真实值面板与搜索轨迹面板配对，前者以固定锚点评分各臂，从而允许直接比较；后者在搜索消耗令牌时跟踪最佳信念效用。圆圈标记任务智能体（写作者、证明者或编码者），下三角标记学习评估器（审稿人或评分者），颜色标识臂。带皇冠的心形标记真实值面板中显示的单一全局最佳信念获胜者。

搜索轨迹面板则绘制每个任务智能体相对于其自身评估器的最佳信念效用，因此高度在不同臂之间不可比；跨臂获胜者从真实值面板和表格中读取。轨迹上的标记是该臂的轮次局部最佳信念选择，之后在留出测试数据或固定评估器上评分。虚线规则上的皇冠标记评估器替换，交替阴影带标记轮次。每次替换时最佳信念可能下降，因为选择性擦除丢弃了依赖于被替换评估器的记录，然后在新、通常更严格的标准下重新攀升。最佳信念为 0 标记轮次开始，此时尚无记录通过新评估器，或智能体彻底失败任务，例如代码无法编译。轨迹在其最佳信念智能体停止变化处结束，不是因为搜索停止（总预算已匹配），而是因为后续智能体超越它；淡虚线显示延续轨迹。每个面板带有两个图例：内部主图例用于被搜索的智能体，上方条带用于基线和配对角色臂。箭头标记每个度量的方向， $\uparrow$ higher-is-better 和 $\downarrow$ lower-is-better；校准轴无优先方向，如接受率和秩相关性，不携带箭头。

5 结果

我们评估了 RQGM，验证了：(i) 协同进化的学习式评估即使在存在真实答案的可验证编码任务上也能改进搜索，以更低的搜索成本超过了 HGM-H 的留出集通过率（第 5.1 节）；(ii) 协同进化在没有任何基准能够对工件进行评分的领域中改进了生成器，产生了更强的论文作者和奥林匹克证明者（第 5.2 节）。第 5.3 节随后考察了效用转换如何塑造搜索。我们接着验证了 (iii) 协同进化改进了评估器本身，强化了评分器并纠正了论文审稿人对人工智能生成文本的过度宽容（第 5.4 节）。最后，我们提供了一个有针对性的消融实验，表明在最终评分模型保持不变的情况下，更低成本的搜索时任务代理调用可以保持论文领域的端点（第 5.5 节）。更多细节和消融实验见附录 C 和 D。

5.1 学习式评估在存在真实答案的领域同样有效 (RQ1)

我们现在展示 RQGM 即使在存在客观真实答案评估的领域也能改进搜索。在多语言编码领域，我们协同进化了一个代码审稿人与编码器。由于多语言编码围绕多轮代理编辑构建，这种代理作为评判者提供了一个更便宜且互补的替代目标。元代理不仅学习补丁是否通过测试，还学习补丁是否具有高质量。RQGM 在 $1.35\times - 1.72\times$ 更低的令牌成本下，对专家和通才均超过了 HGM-H 的留出集通过率（图 1）。检查元代理编辑代码库的位置揭示了协同进化两个角色的价值：在多语言编码运行中，90% 被接受的补丁修改了编码器和审稿人共用的共享任务代理功能或基础设施，而不是角色特定的代码（图 10）。因此，单次这样的编辑同时改进了两个角色，表明协同进化丰富了搜索，而不是将努力分散到相互竞争的目标上。

RQ1: RQGM 是一个通用框架，用于丰富搜索可用的效用信号：即使在存在真实答案的情况下，协同进化的学习式评估器也能提供互补目标，提高搜索效率，并有望应用于评估更慢的领域。

5.2 学习式评估在缺乏客观评估的领域改进代理 (RQ2)

我们现在转向没有客观基准的领域，其中仅存在依赖评估者的任务智能体。我们使用来自本工作及先前基线的广泛审稿人和评分者小组，对论文撰写器和证明器产生的工件进行评分。任务智能体及其评估者在整个搜索过程中交互，但在本节中我们仅评估任务智能体，将评估者推迟到第 5.4 节，那里客观锚定数据允许进一步分析。

论文撰写。 论文质量无法客观评估，因此我们使用来自本工作及先前基线的固定四位审稿人小组（表 1）对撰写器工件进行评分；撰写器与其共同演化的审稿人的联合搜索如图 6 右侧所示。我们的结果表明，将撰写器与学习到的审稿人共同演化可改善其性能。在 HGM-H 提交其撰写器的匹配计算点，我们的撰写器平均已实现 $1.78\times$ 更高的审稿人小组接受率，并且在整个运行过程中，它在每位审稿人的评估下均优于 HGM-H，接受率高出 $1.86\times$ 。这得益于共同演化的审稿人，其目标被正则化以偏好对 AI 生成文本更严格的审稿人（第 5.4 节），从而为撰写器提供更严格且能抵抗奖励黑客行为的信号。

证明撰写。 证明领域比论文撰写困难得多，改进仅在更长的训练时间范围内出现。如表 2 所示，RQGM 通才在匹配计算下表现与 HGM-H 相似。差异出现在更长的时间范围：HGM-H 停滞不前，我们推测这是因为冻结的评估者最终停止提供有信息量的信号。RQGM 发现的专家证明器具有最佳的小组平均分和 Pass@6 分数。正如我们在第 5.4 节所示，这得益于共同演化的评分器，其性能超过 ProofAutoGrader 和 HGM-H，提供的评估能引导搜索找到比 HGM-H 更好的解决方案。

更细致的比较是 Huang 和 Yang 的 IMO25 基线 [52]，这是一个人工设计的验证与精炼流程，在 IMO 2025 中取得了金牌级表现。RQGM 专家获得了更高的小组平均分和 Pass@6 率，但这是通过找到更多近乎完整的证明（7 个中的 6 个）实现的，在更严格的 Pass@7 指标上有所让步。它仍然在 Pass@7 上超过最佳的 HGM-H 证明器，并且已经超过了 IMO25 基线的平均分，尽管使用了

表 1：协同演化的作者在匹配搜索成本下达到比 HGM-H 高出 1.78× 的平均接受率，最佳专业作者则高出 1.86×。行表示作者智能体；四个审稿人列构成来自我们工作及先前基线的固定评审小组，每个单元格给出审稿人接受作者生成论文的百分比（均值 $\pm$ 95% 杰弗里斯区间）。均值在评审小组上取平均，括号内为每个 RQGM 作者相对于 HGM-H 的提升。RQGM 对抗性是对 AI 生成文本更严格的 RQGM 审稿人（第 5.4 节）。每列最佳以粗体显示。

Writer	Search Tokens	Reviewer Acceptance Rate (% , $\uparrow$)				Mean (% , $\uparrow$)
		Sakana [14]	DGM-H [4]	HGM-H [3, 4]	RQGM adversarial	
HGM-H writer	42.5M	1.0 $\pm$ 2.2	12.0 $\pm$ 6.3	64.0 $\pm$ 9.3	10.0 $\pm$ 5.9	21.8 $\pm$ 4.0
RQGM writer (generalist)	44.6M	2.0 $\pm$ 2.9	43.0 $\pm$ 9.6	81.0 $\pm$ 7.6	29.0 $\pm$ 8.8	38.8 $\pm$ 4.8 (1.78 $\times$)
RQGM writer (specialist)	221.8M	5.0 $\pm$ 4.3	40.0 $\pm$ 9.5	86.0 $\pm$ 6.8	31.0 $\pm$ 9.0	40.5 $\pm$ 4.8 (1.86 $\times$)

没有手工设计的奥林匹克特定脚手架。因此，我们认为缩小剩余的 Pass@7 差距取决于增加搜索预算。详情见附录 D。

表 2：协同演化的 RQGM 专业证明器获得最佳平均分，而 HGM-H 证明器和 RQGM 通用证明器低于基线。行表示证明器智能体。来自我们工作及先前基线的固定评分小组对每个证明器的证明进行评分，与表 1 的审稿人小组相对应。由于评分者报告多个度量，我们对每个度量在评分者间进行汇总，每个评分者的结果见表 6。分数是平均等级（0–7），**Pass@6** 是至少获得 7 分中 6 分的比例，**Pass@7** 是获得满分的比例；所有报告 $\pm$ 标准误。每列最佳以粗体显示。

Prover	Search Tokens	Score ($\uparrow$)	Pass@6 ($\uparrow$)	Pass@7 ($\uparrow$)
Static IMO25 prover	N/A	4.07 $\pm$ 0.42	55.0% $\pm$ 6.4	55.0% $\pm$ 6.4
HGM-H prover	21.5M	3.73 $\pm$ 0.43	51.7% $\pm$ 6.5	45.0% $\pm$ 6.4
RQGM prover (generalist)	37.9M	3.73 $\pm$ 0.43	51.7% $\pm$ 6.5	45.0% $\pm$ 6.4
RQGM prover (specialist)	88.0M	4.33 $\pm$ 0.41	61.7% $\pm$ 6.3	48.3% $\pm$ 6.5

RQ2：在没有客观基准的情况下，使用学习评估器协同演化的智能体优于固定评估器基线：作者达到显著更高的审稿人小组接受率，而证明者在更长时间范围内改进，因为协同演化的评分者变得更准确，而冻结的基线停滞不前。

5.3 评估器替换作为搜索的课程（RQ3）

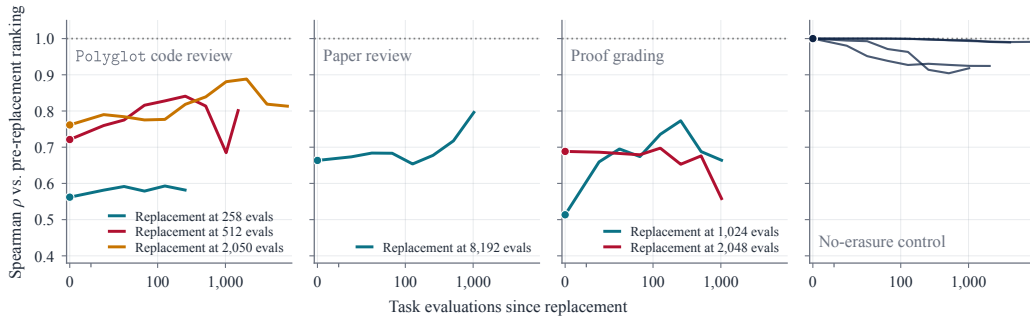


图 3：评估器替换永久性地重新排列档案。每条曲线跟踪一次替换，绘制替换后与替换前排名的斯皮尔曼 ρ 随评估累积的变化： $\rho = 1$ （虚线）是未改变的排序， $\rho = 0$ 是不相关的重新排序。在所有三个任务中 ρ 稳定在远低于 1 的水平且从未恢复，因此新排序得以保持。无擦除控制（最右侧）保持高位（ $\rho \geq 0.90$ ），表明擦除对于效用转换引导搜索是必要的。

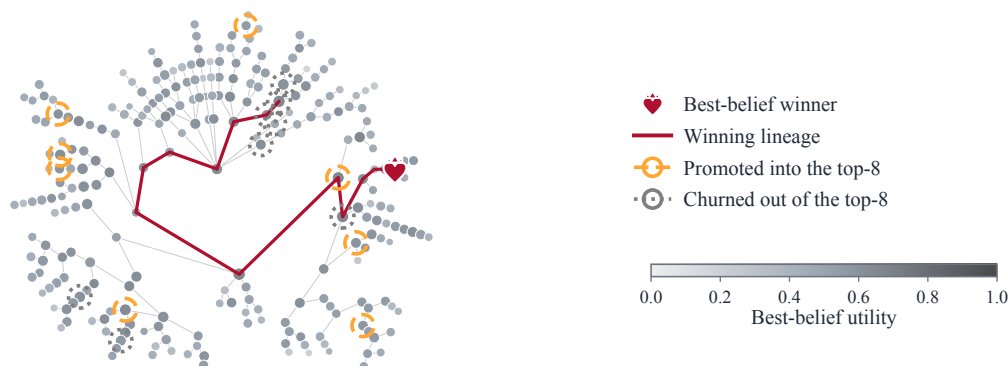


图 4：评估器替换保留了最佳谱系，同时对剩余部分重新排序。评估器替换后的论文运行档案（径向布局；节点颜色表示最佳信念效用，大小反映证据计数）。深红色获胜谱系完整保留，以带皇冠的心形获胜者结束。在前 8 个节点中，每个先前成员都被重新排序（环标记晋升和更替）。

RQGM 改进任务代理的一个潜在机制是，逐步增强的评估器随时间强化种群，对任务代理施加类似课程的效果。要实现这一点，每次转换必须同时达成两个结果：在新的、更严格的标准下重新排序搜索，而不是保留旧顺序，同时仍让强大的谱系延续，使种群随时间前进，而不是在每次转换时重新开始。

宏观视角：每次替换都重新排序档案。如果替换仅仅锐化了通过率估计，而没有改变搜索所偏好的代理，就不会产生课程。图 3 表明并非如此。一些排序结构在转换中持续存在，反映了通过变化传递的评估器无关证据，但重新排序是实质性的且永久的，稳定在擦除后的水平附近，而不是恢复到旧顺序。因此，每个后续评估器似乎都执行了更严格的标准。选择性擦除使这成为可能：保留陈旧分数的无擦除对照保持在被取代的顺序上，从不允许新标准重新排序代理。

微观视角：课程推进骨干谱系。

图 4 提供了更细致的视角，档案将证据集中在少数骨干谱系上。每次转换将被取代标准排名较低的谱系拉入竞争，因此新标准由不同的候选者满足，而不是同一组重新评分。然而，最佳谱系在扩大的集合中保持韧性，表明课程围绕稳定的骨干提升种群，而不是使其分散。

研究问题 3：评估器替换驱动类似课程的搜索：擦除使每个更严格的标准能够将档案重新排序到新候选者，而韧性的骨干保持种群前进。

5.4 协同进化提升评估器自身（研究问题 4）

尽管研究问题生成模型（RQGM）并非旨在直接改进评估器，本文认为，提升生成器的相同动力学机制同样能惠及评估器，因为更深入地理解如何生成解决方案可能有助于元代理对其进行判别。互补的效用信号同样可以防止某一角色陷入停滞时整体停滞。本文学习的两个评估器——证明评分器和论文审稿人——在一个关键方面存在差异：它们对自我偏好偏差的敏感性，即大语言模型评判者倾向于偏好人工智能生成文本的倾向 [16]。评分器在很大程度上不受此影响，仅通过协同进化即可改进；而审稿人则不然，需要受控效用进化所实现的对抗性校正来指导论文写作搜索（第 4 节和表 1）。

证明评分。评分是约束更强的任务：评分器以参考解答及其步骤为条件，因此它依据固定目标而非表面合理性来评判证明。结合基础模型（Foundation Model）的数学理解和指令遵循能力 [55 – 57]，这几乎没有留下自我偏好偏差的空间，使得对抗性目标变得不必要。

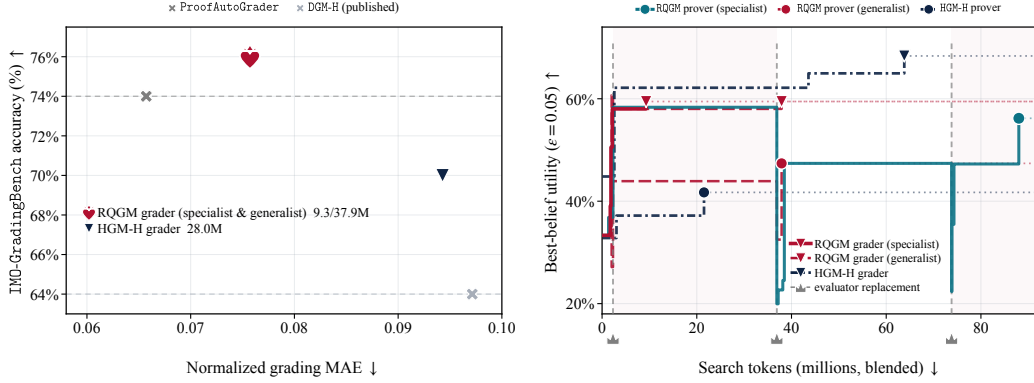


图 5：协同进化的 RQGM 评分器在 3× 更低的搜索成本下达到了 IMO-GradingBench 的最佳

准确率。具有真实答案锚定的槽位有一个全局最佳信念赢家（带皇冠的心形标记），而依赖评估器的槽位仅承认轮次局部赢家，在表格中进行事后评分。左图：所选评分器在 IMO-GradingBench 上的准确率与评分平均绝对误差（MAE）的关系；RQGM 评分器的全局赢家（心形标记）是准确率最高的点，专家型与通用型评分器重合。右图：以令牌计的搜索轨迹，显示评分器（上）和证明器（下）的最佳信念效用；评分器依据其真实答案锚定进行评分，因此其高度具有可比性并包含全局赢家，而每个证明器标记仅是其自身评估器下该轮次的最佳信念赢家。每次评估器替换（皇冠标记；阴影轮次）时效用下降，因为擦除操作丢弃了被替换的记录，随后重新攀升；线条终止于其最佳信念代理停止改进之处，而非搜索停止之处。更多细节参见第 4 节和第 4.1 节。

如图 5 所示，协同进化的评分器是最强的评判者，优于静态基线和最佳的 HGM-H 评分器，并且其搜索成本更低。与其他实验不同，专家评分器和通用评分器在单一节点上重合，该节点在通用选择之前先达到专家选择。在专家选择点，其令牌效率比 HGM-H 高 3×；即使在稍后的通用选择点，其成本高出 1.35×，它仍然击败了 HGM-H 所能找到的最佳评分器，因为 HGM-H 在相同的搜索预算内从未达到同等水平。因此，协同进化以更低的成本产生了更好的评分器，并让评分器驱动表 2 中的证明器改进，而无需对抗性目标。

论文评审。 论文评审是相反的情况。没有参考答案，因此审稿人根据论文自身条件进行评判，这暴露了大型语言模型评判者的一个已知弱点，即自我偏好偏差：倾向于比人类撰写的文本更容易接受人工智能生成的文本 [16, 58]。APReS 基准加剧了这一问题，因为其接受/拒绝平衡奖励宽松的审稿人，所以审稿人对人工智能生成文本的偏差与其原始 APReS 准确率朝着同一方向拉动：表 1 中为作者打分的 HGM-H 审稿人接受人工智能生成论文的比率是人类论文的 1.42× - 1.91× 倍。

Sakana 基线显示了相反的失败：即使使用真实的 NeurIPS 表格并保持固定，也过于苛刻，为其指导的 HGM-H 作者提供了微弱的信号。我们需要的不是为了准确而准确的审稿人，而是一个能为其协同进化的开放式作者提供强大、难以破解且随时间推移变得更加严格的信号的审稿人。这证明了将目标从原始 APReS 准确率上修改开是合理的。

RQGM 允许我们利用轮次边界来纠正大型语言模型的自我偏好偏差。每次替换后，被取代的审稿人所接受的人工智能生成论文形成一个对抗池，下一个轮次额外奖励进化后的审稿人拒绝这些论文。然后，搜索选择一个对人工智能生成文本特别严格的审稿人，这在构造上牺牲了原始准确率，使我们找到的最佳审稿人低于宽松的 HGM-H。它获得的是校准：审稿人以相似的比率接受人工智能生成和真实论文，同时保持 80% 的真实准确率。如果评估器和生成器独立进化，作者可能会通过奖励黑客手段利用它，因此协同进化在人类生成文本和人工智能生成文本之间产生了有意义的决策边界。

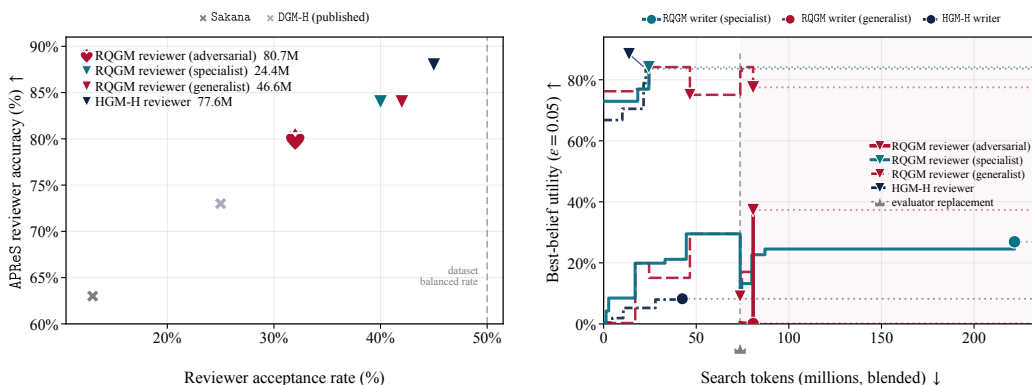


图 6：对抗性 RQGM 审稿人以相近的比率接受 AI 论文和人类论文，这一校准后的接受/拒绝边界驱动了最强的写作者（表 1）；HGM-H 仅通过过度接受 AI 生成的论文才达到更高的原始 APReS 准确率，这使其写作者表现较弱。左图：所选审稿人的 APReS 准确率与接受率的关系；虚线标记了数据集的真实接受率。RQGM 审稿人在低接受率下保持高准确率，介于宽松的 HGM-H 和过于严苛的 Sakana 之间。右图：同一运行过程按令牌展示，写作者（上）和审稿人（下）；在对抗池替换时，通才的平均最佳信念下降，因为擦除重新排序了受影响的效用，然后在更严苛的标准下重新攀升。

RQ4：改进生成器的相同搜索动态也能改进评估器：协同进化以低于固定评估器基线的成本找到更强的评分器，并且通过在轮次边界引入对抗性目标，纠正了审稿人的自我偏好偏差，为写作者提供了更难被攻击的信号。

5.5 通过混合模型方法降低搜索成本

在任务和评估器角色之间共享扩展提供了 RQGM 的部分效率（图 10），但扩展调用仅占我们所有领域总搜索成本的 $\approx 20\%$ （图 9）。为了进一步降低成本，我们考虑一种混合模型方法：保持元代理为 GPT-5.5（低），但将搜索时的任务代理调用路由到更快且成本更低的 Nemotron 3 Ultra [59]。

价格等价核算。如图 7 所示，将任务代理从 GPT-5.5 替换为 Nemotron 3 Ultra，在相同最终评估下，搜索令牌成本降低了 $\approx 13.0\times$ ，同时接近仅使用 GPT-5.5 运行的准确率。我们预期这一优势具有领域特定性，取决于成本较低的任务代理是否提供有信息量的信号。如果任务代理的基础模型在某个领域能力不足，例如构建新颖证明，则可能增加搜索成本，因为不同的元代理扩展在质量上难以区分。

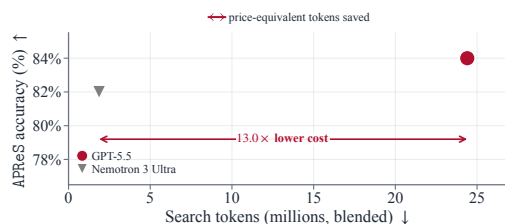


图 7：Nemotron 3 Ultra 任务代理调用以 $\approx 13.0\times$ 更低的搜索令牌成本接近仅使用

GPT-5.5 的运行。在留下的 APReS 分割上，最终的 GPT-5.5 评估显示，Nemotron 3 Ultra 发现的框架接近我们仅使用 GPT-5.5 框架的性能。由于原始令牌不可直接比较，我们根据撰写时可获得的最佳定价报告 GPT-5.5 价格等价的混合令牌。扩展调用按与仅使用 GPT-5.5 运行相同的费率计费（附录 C.4）。

成本消融：经验上，RQGM 搜索的主要成本驱动因素是评估而非扩展。在论文评审中，通过更高效的基础模型路由搜索时的任务代理调用，可大幅降低搜索成本，同时最终性能损失可接受；这一权衡在其他领域仍有待研究。

6 结论

我们提出了红皇后哥德尔机 (RQGM)，一个评估者与其评分的代理共同演化的框架。我们的结果揭示了两个广泛原则。第一，评估者与生成器共同演化，能够在难以验证的任务（如论文写作和证明写作）上实现改进，而固定基准无法直接对工件评分，这反映了自然界中观察到的共同演化动态 [1]。第二，共同演化系统达到或超过固定评估者基线，同时通常更具令牌效率，我们推测这归因于共享扩展成本、更多样化的效用信号，以及一种课程式效应——逐渐严格的评估者随时间强化种群。两者共同的基础是框架对非平稳效用的适应：由于受控的效用演化允许搜索目标在训练轮次边界变化，同时保持每轮次的保证，固定目标无法表达的干预（例如纠正审稿人自我偏好偏差的对抗项）在搜索过程中变得可用。

6.1 未来研究方向

我们的研究是初步的，基于我们计划扩展的短搜索视野。该框架的前提是，更长的共同进化会加剧这些效应，扩展课程，使每一代智能体面对更严格、更具辨别力的评判者。这种改进的极限由锚点决定。将每次替换锚定到固定的真实值上，可以保持评估器的准确性，但将其限制在锚点自身的决策边界内。因此，远超基准的改进必须来自叠加在真实值信号之上的目标，而锚点则作为防止漂移的护栏机制。对抗性审稿人是第一个此类实例：它在对抗项发现基准从未指定的人机决策边界的同时，大致保持了在真实值上的准确率。我们相信，这类更丰富的目标是系统超越静态基准、自我引导评估的途径。我们指出了当前框架的两个局限性。首先，评估器的质量仅与其锚点相当：一个弱或有偏的锚点可能产生无信息量的评估器。其次，我们的理论保证是训练轮次局部的，仅涵盖单个训练轮次内的改进。我们在附录 G 中全面讨论了 RQGM 的局限性，并将在探索这些未来研究方向时更新本工作。

致谢

本研究得到了以下实体的支持：NVIDIA、英国皇家工程院通过 DANTE (RAEng 主席项目) 提供的资助；欧洲研究理事会，特别是 REDIAL 项目；SPRIND 在复合学习挑战下的资助；Foresight Institute 通过 AI 安全基金提供的资助；谷歌通过谷歌学术研究奖提供的资助。

参考文献

- [1] Leigh Van Valen. A new evolutionary law. *Evolutionary Theory*, 1:1–30, 1973.
- [2] Jenny Zhang, Shengran Hu, Cong Lu, Robert Tjarko Lange, and Jeff Clune. Darwin Gödel Machine: Open-ended evolution of self-improving agents. In *ICLR*, 2026. URL <https://openreview.net/forum?id=pUpzQZTvGY>.
- [3] Wenyi Wang, Piotr Piękos, Li Nanbo, Firas Laakom, Yimeng Chen, Mateusz Ostaszewski, Mingchen Zhuge, and Jürgen Schmidhuber. Huxley-Gödel Machine: Human-Level Coding Agent Development by an Approximation of the Optimal Self-Improving Machine. In *ICLR*, 2026. URL <https://openreview.net/forum?id=T0EiEuhOOL>.
- [4] Jenny Zhang, Bingchen Zhao, Wannan Yang, Jakob N. Foerster, Jeff Clune, Minqi Jiang, Sam Devlin, and Tatiana Shavrina. HyperAgents. *CoRR*, abs/2603.19461, 2026.
- [5] Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob N. Foerster, Jeff Clune, and David Ha. The AI scientist: Towards fully automated open-ended scientific discovery. *CoRR*, abs/2408.06292, 2024.
- [6] Yutaro Yamada, Robert Tjarko Lange, Cong Lu, Shengran Hu, Chris Lu, Jakob N. Foerster, Jeff Clune, and David Ha. The AI scientist-v2: Workshop-level automated scientific discovery via agentic tree search. *CoRR*, abs/2504.08066, 2025.

- [7] Thang Luong, Dawsen Hwang, Hoang H. Nguyen, Golnaz Ghiasi, Yuri Chervonyi, Insuk Seo, Junsu Kim, Garrett Bingham, Jonathan Lee, Swaroop Mishra, Alex Zhai, Clara Huiyi Hu, Henryk Michalewski, Jimin Kim, Jeonghyun Ahn, Junhwi Bae, Xingyou Song, Trieu H. Trinh, Quoc V. Le, and Junehyuk Jung. Towards robust mathematical reasoning. In *EMNLP*, pages 35418–35442. Association for Computational Linguistics, 2025.
- [8] Sergey Levine, Peter Pastor, Alex Krizhevsky, Julian Ibarz, and Deirdre Quillen. Learning hand-eye coordination for robotic grasping with deep learning and large-scale data collection. *Int. J. Robotics Res.*, 37(4-5):421–436, 2018.
- [9] Benjamin Burger, Phillip M. Maffettone, Vladimir V. Gusev, Catherine M. Aitchison, Yang Bai, Xiaoyan Wang, Xiaobo Li, Ben M. Alston, Buyi Li, Rob Clowes, Nicola Rankin, Brandon Harris, Reiner Sebastian Sprick, and Andrew I. Cooper. A mobile robotic chemist. *Nature*, 583(7815):237–241, 2020. doi: 10.1038/s41586-020-2442-2.
- [10] Marcin Andrychowicz, Filip Wolski, Alex Ray, Jonas Schneider, Rachel Fong, Peter Welinder, Bob McGrew, Josh Tobin, OpenAI Pieter Abbeel, and Wojciech Zaremba. Hind-sight experience replay. In I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, editors, *NeurIPS*, volume 30. Curran Associates, Inc., 2017. URL https://proceedings.neurips.cc/paper_files/paper/2017/file/453fadbd8a1a3af50a9df4df899537b5-Paper.pdf.
- [11] Haoyan Yang, Mario Xerri, Solha Park, Huajian Zhang, Yiyang Feng, Sai Akhil Kogilathota, and Jiawei Zhou. Self-improvement of large language models: A technical overview and future outlook. *CoRR*, abs/2603.25681, 2026.
- [12] Adriano Barbosa-Silva, Simon Ott, Kathrin Blagec, Jan Brauner, and Matthias Samwald. Mapping global dynamics of benchmark creation and saturation in artificial intelligence. *CoRR*, abs/2203.04592, 2022.
- [13] Paul Gauthier. o1 tops aider’s new polyglot leaderboard. <https://aider.chat/2024/12/21/polyglot.html>, December 2024. Accessed: 2026-01-28.
- [14] Chris Lu, Cong Lu, Robert Tjarko Lange, Yutaro Yamada, Shengran Hu, Jakob Foerster, David Ha, and Jeff Clune. Towards end-to-end automation of AI research. *Nature*, 651(8107):914–919, 2026.
- [15] Bingchen Zhao, Jenny Zhang, Chenxi Whitehouse, Minqi Jiang, Michael Shvartsman, Abhishek Charnalia, Despoina Magka, Tatiana Shavrina, Derek Dunfield, Oisin Mac Aodha, and Yoram Bachrach. APReS: an agentic paper revision and evaluation system. *CoRR*, abs/2603.03142, 2026.
- [16] Arjun Panickssery, Samuel R. Bowman, and Shi Feng. LLM evaluators recognize and favor their own generations. In *NeurIPS*, 2024.
- [17] Irving John Good. Speculations concerning the first ultraintelligent machine. *Adv. Comput.*, 6: 31–88, 1965.
- [18] Jürgen Schmidhuber. Gödel machines: Self-referential universal problem solvers making provably optimal self-improvements. *CoRR*, cs.LO/0309048, 2003.
- [19] Louis Kirsch and Jürgen Schmidhuber. Eliminating meta optimization through self-referential meta learning. *CoRR*, abs/2212.14392, 2022.
- [20] Chris Lu, Sebastian Towers, and Jakob N. Foerster. Arbitrary order meta-learning with simple population-based evolution. *CoRR*, abs/2303.09478, 2023.
- [21] Chunqiu Steven Xia, Zhe Wang, Yan Yang, Yuxiang Wei, and Lingming Zhang. Live-SWE-agent: Can software engineering agents self-evolve on the fly? *CoRR*, abs/2511.13646, 2025.
- [22] Eric Zelikman, Eliana Lorch, Lester Mackey, and Adam Tauman Kalai. Self-taught optimizer (STOP): recursively self-improving code generation. *CoRR*, abs/2310.02304, 2023.

- [23] Xunjian Yin, Xinyi Wang, Liangming Pan, Li Lin, Xiaojun Wan, and William Yang Wang. Gödel agent: A self-referential agent framework for recursively self-improvement. In *ACL (1)*, pages 27890–27913. Association for Computational Linguistics, 2025.
- [24] Chrisantha Fernando, Dylan Banarse, Henryk Michalewski, Simon Osindero, and Tim Rocktäschel. Promptbreeder: Self-referential self-improvement via prompt evolution. In *ICML*, volume 235 of *Proceedings of Machine Learning Research*, pages 13481–13544. PMLR / OpenReview.net, 2024.
- [25] Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah D. Goodman. Star: Bootstrapping reasoning with reasoning. In *NeurIPS*, 2022.
- [26] William F. Shen, Alex Jacob, Zichen Zhang, Daoheng Wang, Wenyi Wang, Rui Liang, Yulong Zhang, Xinchu Qiu, and Nicholas D. Lane. Star analyst: Self-tuning alpha research. SSRN preprint, 2026. Available at SSRN: <https://doi.org/10.2139/ssrn.6823940>.
- [27] Maxime Robeyns, Martin Szummer, and Laurence Aitchison. A self-improving coding agent. *CoRR*, abs/2504.15228, 2025.
- [28] Joar Skalse, Nikolaus H. R. Howe, Dmitrii Krashennnikov, and David Krueger. Defining and characterizing reward gaming. In *NeurIPS*, 2022.
- [29] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-judge with MT-bench and chatbot arena. In *NeurIPS*, 2023.
- [30] Yann Dubois, Percy Liang, and Tatsunori Hashimoto. Length-controlled AlpacaEval: A simple debiasing of automatic evaluators. In *First Conference on Language Modeling*, 2024. URL <https://openreview.net/forum?id=CybBmzWBX0>.
- [31] Mingchen Zhuge, Changsheng Zhao, Dylan R. Ashley, Wenyi Wang, Dmitrii Khizbullin, Yunsong Xiong, Zechun Liu, Ernie Chang, Raghuraman Krishnamoorthi, Yuandong Tian, Yangyang Shi, Vikas Chandra, and Jürgen Schmidhuber. Agent-as-a-judge: Evaluate agents with agents. In *ICML*, volume 267 of *Proceedings of Machine Learning Research*. PMLR / OpenReview.net, 2025.
- [32] Juraj Gottweis, Wei-Hung Weng, Alexander Daryin, Tao Tu, Petar Sirkovic, Artiom Myaskovsky, Grzegorz Glowaty, Felix Weissenberger, Alessio Orlandi, Dan Popovici, et al. Accelerating scientific discovery with Co-Scientist. *Nature*, pages 1–3, 2026.
- [33] Ali Essam Ghareeb, Benjamin Chang, Ludovico Mitchener, Angela Yiu, Caralyn J Szostkiewicz, Dmytro Shved, Gavin J Gyimesi, Jon M Laurent, Samantha M Wright, Muhammed T Razzak, et al. A multi-agent system for automating scientific discovery. *Nature*, pages 1–3, 2026.
- [34] Alexander Novikov, Ngân Vu, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, M. Pawan Kumar, Abigail See, Swarat Chaudhuri, George Holland, Alex Davies, Sebastian Nowozin, Pushmeet Kohli, and Matej Balog. AlphaEvolve: A coding agent for scientific and algorithmic discovery. *CoRR*, abs/2506.13131, 2025.
- [35] Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M. Pawan Kumar, Emilien Dupont, Francisco J. R. Ruiz, Jordan S. Ellenberg, Pengming Wang, Omar Fawzi, Pushmeet Kohli, and Alhussein Fawzi. Mathematical discoveries from program search with large language models. *Nature*, 625(7995):468–475, 2024.
- [36] Christopher D. Rosin and Richard K. Belew. New methods for competitive coevolution. *Evol. Comput.*, 5(1):1–29, 1997.
- [37] Jürgen Schmidhuber and Jieyu Zhao. Multi-agent learning with the success-story algorithm. In *ECAI Workshop LDAIS/ICMAS Workshop LIOME*, volume 1221 of *Lecture Notes in Computer Science*, pages 82–93. Springer, 1996.

- [38] David Silver, Aja Huang, Chris J Maddison, Arthur Guez, Laurent Sifre, George Van Den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, et al. Mastering the game of Go with deep neural networks and tree search. *Nature*, 529(7587): 484–489, 2016.
- [39] David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dhharshan Kumaran, Thore Graepel, Timothy Lillicrap, Karen Simonyan, and Demis Hassabis. A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play. *Science*, 362(6419):1140–1144, 2018. doi: 10.1126/science.aar6404.
- [40] Yixing Chen, Yiding Wang, Siqi Zhu, Haoifei Yu, Tao Feng, Muhan Zhang, Mostofa Patwary, and Jiaxuan You. Multi-agent evolve: LLM self-improve through co-evolution. *CoRR*, abs/2510.23595, 2025.
- [41] Zhaotian Weng, Antonis Antoniadis, Deepak Nathani, Zhen Zhang, Xiao Pu, and Xin Eric Wang. Group-evolving agents: Open-ended self-improvement via experience sharing. *CoRR*, abs/2602.04837, 2026.
- [42] Jason Weston and Jakob N. Foerster. AI & human co-improvement for safer co-superintelligence. *CoRR*, abs/2512.05356, 2025.
- [43] Kenneth O. Stanley, Joel Lehman, and Lisa Soros. Open-endedness: The last grand challenge you’ve never heard of. O’Reilly Radar (online article), December 2017. URL <https://www.oreilly.com/radar/open-endedness-the-last-grand-challenge-youve-never-heard-of/>. Accessed 2026-06-16.
- [44] Edward Hughes, Michael D. Dennis, Jack Parker-Holder, Feryal M. P. Behbahani, Aditi Mavalankar, Yuge Shi, Tom Schaul, and Tim Rocktäschel. Position: Open-endedness is essential for artificial superhuman intelligence. In *ICML*, volume 235 of *Proceedings of Machine Learning Research*, pages 20597–20616. PMLR / OpenReview.net, 2024.
- [45] Jean-Baptiste Mouret and Jeff Clune. Illuminating search spaces by mapping elites. *CoRR*, abs/1504.04909, 2015.
- [46] Adrien Ecoffet, Joost Huizinga, Joel Lehman, Kenneth O. Stanley, and Jeff Clune. Go-explore: a new approach for hard-exploration problems. *CoRR*, abs/1901.10995, 2019.
- [47] Shengran Hu, Cong Lu, and Jeff Clune. Automated design of agentic systems. In *ICLR*. OpenReview.net, 2025.
- [48] Jeff Clune. Ai-gas: Ai-generating algorithms, an alternate paradigm for producing general artificial intelligence. *CoRR*, abs/1905.10985, 2019.
- [49] Yizao Wang, Jean-yves Audibert, and Rémi Munos. Algorithms for infinitely many-armed bandits. In D. Koller, D. Schuurmans, Y. Bengio, and L. Bottou, editors, *NeurIPS*, volume 21, pages 1729–1736. Curran Associates, Inc., 2008.
- [50] Li Zhang. CRAVE: Code Review Agent Verdict Evaluation. Hugging Face dataset, 2025. URL <https://huggingface.co/datasets/TuringEnterprises/CRAVE>. Code review classification dataset from curated human code reviews.
- [51] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R. Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *ICLR*. OpenReview.net, 2024.
- [52] Yichen Huang and Lin F. Yang. Gemini 2.5 pro capable of winning gold at IMO 2025. *CoRR*, abs/2507.15855, 2025.
- [53] OpenAI. Api pricing. <https://openai.com/api/pricing/>, 2026.
- [54] Anthropic. Pricing. <https://platform.claude.com/docs/en/about-claude/pricing>, 2026.

- [55] OpenAI. Our First Proof Submissions. <https://openai.com/index/first-proof-submissions/>, 2025. Gold-medal-level performance at the International Mathematical Olympiad 2025.
- [56] Sébastien Bubeck, Christian Coester, Ronen Eldan, Timothy Gowers, Yin Tat Lee, Alexandru Lupsasca, Mehtaab Sawhney, Robert Scherrer, Mark Sellke, Brian K. Spears, Derya Unutmaz, Kevin Weil, Steven Yin, and Nikita Zhivotovskiy. Early science acceleration experiments with GPT-5. *CoRR*, abs/2511.16072, 2025.
- [57] Jeffrey Zhou, Tianjian Lu, Swaroop Mishra, Siddhartha Brahma, Sujoy Basu, Yi Luan, Denny Zhou, and Le Hou. Instruction-following evaluation for large language models. *CoRR*, abs/2311.07911, 2023.
- [58] Fengqing Jiang, Yichen Feng, Yuetai Li, Luyao Niu, Basel Alomair, and Radha Poovendran. BadScientist: Can a research agent write convincing but unsound papers that fool LLM reviewers? *CoRR*, abs/2510.18003, 2025.
- [59] NVIDIA. Nemotron 3 ultra: Open, efficient mixture-of-experts hybrid mamba-transformer model for agentic reasoning. *CoRR*, abs/2606.15007, 2026.
- [60] NVIDIA. Nemotron 3 super: Open, efficient mixture-of-experts hybrid mamba-transformer model for agentic reasoning. *CoRR*, abs/2604.12374, 2026.
- [61] DeepInfra. Pricing. <https://deepinfra.com/>, 2026.

目录

A 附录概览 18

B 完整算法 18

C 实验设置	19
C.1 领域与锚点对	19
C.2 模型与运行标识	20
C.3 搜索与调度设置	20
C.4 留出评估协议	20
C.5 初始智能体与提示	22

D 结果与消融	24
D.1 原始留出计数	24
D.2 机制消融	24

D.3 成本与进展的来源	25
--------------	----

E 共同进化发现了什么	26
E.1 进化后的评估器改变了什么	26
E.2 共同进化的评分器	26
E.3 多语言代码审稿人	27
E.4 获胜谱系	27

F 理论	30
F.1 设定: HGM 搜索算法	30
F.2 替换与选择的一条规则	31
F.3 工作后验一致性	31
F.4 轮次局部固定效用有效性	33
F.5 受控效用演化的支撑性结果	36

G 局限性 38

附录概览

本附录涵盖 RQGM 算法（附录 B）、实验细节（附录 C）、实验结果（附录 D）、元代理如何修改代码（附录 E）、理论（附录 F）以及本工作的局限性（附录 G）。

B 完整算法

[Algorithm 1](#) states the full RQGM procedure of [Sec. 3](#). Lines 1–4 initialize the frozen-epoch archive from the seed workspace a_0 : every evaluator slot is set to its first epoch, frozen on its incumbent, and the seed is train-evaluated for lineage evidence only. Lines 5–21 run the inherited CMP-guided search between checkpoints. At each iteration, the expansion gate (line 9) either expands or evaluates: an

Algorithm 1: Red Queen Gödel Machine search with anchor-only evaluator replacement.

Input: Seed workspace a_0 ; roles $\mathcal{R}$; validation budget B ; evaluator slots $1:M$

Input: Anchor sets $GT[m]$; checkpoints C ; caps ($\bar{B}_{\text{exp}}, \bar{B}_{\text{train}}, \bar{B}_{\text{val}}$); best-belief level ϵ

Output: Archive $\mathcal{T}_B$ and selected endpoint agent a^*

```
1 Initialize the frozen-epoch archive.
2  $\mathcal{T} \leftarrow \{a_0\}$ ;  $N \leftarrow 0$ ;  $j[m] \leftarrow 1$  for all slots  $m$ 
3  $E[m] \leftarrow \text{Freeze}(\text{Incumbent}(m))$  for all slots  $m$ 
4  $\text{TrainEval}(a_0, \mathcal{R}, E, \bar{B}_{\text{train}})$  // lineage evidence only
5 Run CMP-guided search between checkpoints.
6 while  $N < B$  do
7    $b \leftarrow \min\{c \in C : c > N\} \wedge B$  // next checkpoint
8   while  $N < b$  do
9     if  $\text{ExpandGate}(\mathcal{T}, N)$  then
10       $a_p \leftarrow \text{SampleCladeByCMP}(\mathcal{T})$ 
11       $a' \leftarrow \text{EditWorkspace}(a_p, \mathcal{T}, \bar{B}_{\text{exp}})$ 
12      if  $\text{Valid}(a')$  then
13         $\mathcal{T} \leftarrow \mathcal{T} \cup \{a'\}$ 
14         $\text{TrainEval}(a', \mathcal{R}, E, \bar{B}_{\text{train}})$  // not search utility
15      end
16    end
17     $a \leftarrow \text{SampleCladeByCMP}(\mathcal{T})$  // node-level HGM choice
18     $(r, d) \leftarrow \text{LeastMeasuredCell}(a, \mathcal{R})$  // role/task balancing
19     $o \leftarrow \text{ValEval}(a, r, d, E, \bar{B}_{\text{val}})$ 
20     $\text{AddUtilityRecord}(\mathcal{T}, a, r, d, o, j)$ ;  $N \leftarrow N + 1$ 
21  end
22 At checkpoint, replace evaluators only by anchor best-belief.
23 foreach evaluator slot  $m$  do
24    $\Gamma_m \leftarrow \{E[m]\} \cup \text{Challengers}(m, \mathcal{T})$ 
25    $\text{BB}[e] \leftarrow I_e^{-1}(1 + S_e^{\text{gt}}, 1 + F_e^{\text{gt}})$  for each  $e \in \Gamma_m$ 
26    $e^* \leftarrow \text{Best}(\Gamma_m, \text{BB}; \text{tie} = E[m])$ 
27   if  $e^* \neq E[m]$  then
28      $j[m] \leftarrow j[m] + 1$ ;  $E[m] \leftarrow \text{Freeze}(e^*)$ 
29      $\text{EraseSlotRecords}(\mathcal{T}, m)$ ;  $\text{RecomputeCMPStats}(\mathcal{T})$ 
30   end
31 end
32 end
33 Select the endpoint by best-belief over archive nodes.
34  $a^* \leftarrow \text{BestBeliefNode}(\mathcal{T}, \epsilon)$ 
35 return  $(\mathcal{T}_B, a^*)$ 
```

扩展阶段按元生产力 (metaproductivity) 采样一个父分支 (第 10 行), 将其工作区编辑为子分支 (第 11 行), 并将有效的子分支连同其仅基于谱系的训练评估纳入档案 (第 12–15 行); 随后迭代按 CMP 采样一个节点 (第 17 行), 选取其测量次数最少的角色–任务单元 (第 18 行), 在冻结的评估器下对该单元消耗一个单位的验证预算 (第 19 行), 并记录二元结果 (第 20 行)。第 22–31 行仅在检查点运行, 这是评估器唯一可能发生变化的时刻。对于每个槽位, 现任者及其挑战者通过 ϵ -最佳置信下界在评估器无关锚点计数上进行评分 (第 24–25 行), 得分最高者晋升, 平局时现任者获胜 (第 26 行); 若发生实际变化 (第 27 行), 该槽位推进其训练轮次, 冻结新评估器, 清除被替换槽位的记录, 并重新计算元生产力统计量 (第 28–29 行)。最后, 第 33–35 行返回具有最高 ϵ -最佳置信评分的档案节点。

C 实验设置

C.1 领域与锚点对

图 8 展示了每个领域如何将生成器角色与学习到的评估器角色及真实锚点对。

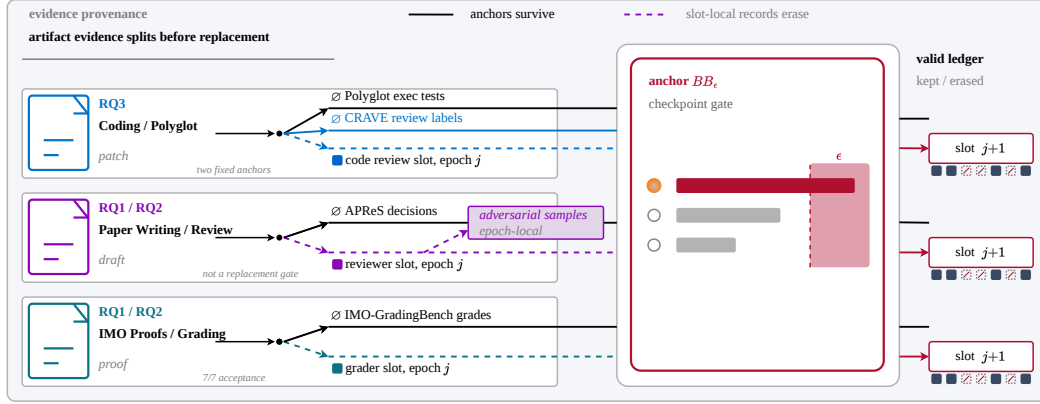


图 8：每个领域将持久的锚点证据与训练轮次局部的评估器记录分离。左：固定的真实锚点（实线）在评估器替换后仍然有效，而训练轮次冻结的评估器槽位记录（虚线）仅属于训练轮次 j_m 。中：只有评估器无关的锚点证据进入 ϵ -最佳置信评估器替换门控。右：替换后，锚点保持有效，而被替换评估器槽位的记录在训练轮次 $j_m + 1$ 之前被清除。

C.2 模型与运行标识

论文主实验、留出评估和面板评分调用中的每次搜索运行均使用 GPT-5.5（低配版）。第 5.5 节的成本削减消融仅改变搜索时的任务代理路由：元代理保持 GPT-5.5（低配版），论文领域的任务代理调用在搜索期间使用 Nemotron 3 Ultra，最终端点评分使用与主论文相同的 GPT-5.5（低配版）审稿人。Nemotron 3 Super [60] 仅用于附录 D.2 的机制消融。我们展示混合令牌总量：训练时生成和评估调用与搜索时效用评估一并统计（第 3.2 节）。总体而言，共有八个主实验，每个领域每个基线一个：编码（RQGM 和 HGM-H）、论文写作（一次 RQGM 运行产生写作者、审稿人和对抗性批评者，加上 HGM-H 审稿人和 HGM-H 写作者）以及证明（一次 RQGM 运行产生证明者和评分者，加上 HGM-H 评分者和 HGM-H 证明者）。每个任务运行到相同的总预算；HGM-H 获得相同的探索预算，通常更早达到平台期。每个代理在其搜索达到最佳信念选择时被读取。我们允许更强大的模型将令牌处理委托给更便宜的模型。对于本文，我们给予 GPT-5.5（低配版）访问 Nemotron 3 Super 的权限；然而，我们从未观察到该功能被使用。类似地，我们为扩展和评估设置了宽松的预算上限作为安全措施，元代理知道这些上限，这可能改变其行为；然而，我们从未观察到单个节点接近这些上限。更严格的上限可能导致早期评估/扩展终止。

C.3 搜索与调度设置

搜索运行附录 F.1 的 HGM 分支元生产力搜索，采用冻结评估器槽位和仅检查点替换；替换和最佳信念选择共享附录 F.2.1 的 ϵ -最佳信念规则，应用于至少累积 5 次评估的节点。表 3 列出固定搜索配置，表 4 列出元代理可编辑表面及其初始值。任何未列出的超参数均取自 Wang 等人 [3]。为确定搜索预算，我们考虑了 Wang 等人 [3] 报告的单个任务 8,000 次评估预算，在将搜索预算翻倍用于我们的两个任务对（16,000）和使用与单个任务相同的预算之间选择了经济适中的方案，最终确定为 $12,288 = 8,192 + 4,096$ 。

C.4 留出评估协议

编码 / 多语言。生成的补丁通过在 166 个留出任务上执行来检查。表 5 列出了计数。

表 3：八个主要运行共享的固定搜索配置。这些常量在每次运行中保持不变，且不暴露给元代理。不使用搜索冷却。

Setting	Value
<i>Models and budget</i>	
Base model	GPT-5.5 (low)
Ablation model	Nemotron 3 Super
Total budget per run	12,288 evaluations
Train samples per node	3
<i>Selection and scheduling</i>	
Best-belief quantile	$\epsilon = 0.05$, five-outcome anchor minimum
Checkpoint schedule	power-of-two ($\rho = 2$)
UCB-Air expansion exponent	$\alpha = 0.6$ (inherited, 3)
Expansion gate	expand when $N_t^\alpha \geq \mathcal{T}_t $
Exploration–exploitation scheduler	B/b (b = remaining budget)
<i>Data splits</i>	
Test sets (APReS/ IMO-GradingBench/ CRAVE)	100 items each
Polyglot train split	10 items
Polyglot validation split	49 items
Polyglot test split	166 items
<i>Budget caps</i>	
Expand cap	\$25, 1,200 s
Train cap	\$8, 900 s
Validation cap	\$25, 1,200 s

表 4：元代理可编辑的界面及其初始值。以下每个设置都暴露给元代理，元代理可以在搜索过程中修改它。

Setting	Initial value
Output tokens per call	32,768
Timeout, run	86,400 s
Timeout, eval	1,200 s
Timeout, LLM	300 s
Timeout, shell	120 s
Coder tool calls per task	≤ 16
Meta-agent tools	persistent bash (120 s); editor; delegation
Meta-agent tool calls per turn	≤ 40

论文写作/审稿。审稿人槽位锚定于 APReS 会议决策；第一个训练轮次在 APReS 决策上训练审稿人并将其冻结为写作者搜索时的批评者，而替换本身仅锚定于 APReS。审稿人最佳信念代理在 100 篇留下的 APReS 论文上根据准确率和接受率进行评分（表 5）；写作者最佳信念代理由表 1 中的固定四位审稿人小组对每位写作者生成的 100 篇论文进行评分。

IMO 评分/证明写作。评分者槽位锚定于 IMO-GradingBench 人工评分；评分者最佳信念代理在 100 个留出项目上根据精确匹配和归一化平均绝对误差进行评分。证明者为 20 道 IMO 问题各写证明，由固定三位评分者小组评分，每个评分者单元格见表 6。Pass@6 统计至少获得 7 分中 6 分的证明；Pass@7 要求满分，并且是相对于静态基线的公开权衡（第 5 节）。

不确定性估计。主文表格中的不确定性汇总了原始结果。对于作者小组（表 1），每个审稿人单元格报告其 $N = 100$ 篇论文结果的 95% 杰弗里斯半宽，而均值列汇总所有 $N = 400$ 篇论文 $\times$ 位

审稿人结果以计算其自身的杰弗里斯半宽。对于证明者小组（表 2），每个单元格汇总 $N = 60$ 个原始结果（20 个问题 $\times$ 3 位评分者）：得分（0–7）为均值 $\pm$ 标准误，Pass@6 / Pass@7 为汇总通过率 $\pm$ s.e.m. $\left(\frac{\epsilon}{\sqrt{\hat{p}(1-\hat{p})/N}} \right)$ 。因此，作者小组单元格为 95% 区间，证明者小组单元格为单个

标准误（约 68% 覆盖率），故两组间的宽度不可直接比较。

成本降低消融。Nemotron 3 Ultra 消融（第 5.5 节）复用附录 C.4 的论文写作/评审协议。搜索时的任务代理调用使用 Nemotron 3 Ultra，而最终 APReS 审稿人端点使用 GPT-5.5（低）评分，采用相同的保留 $N = 100$ APReS 集。代理选择使用 ϵ -最佳信念与 $\epsilon = 0.05$ 。图 7 中的搜索成本以 GPT-5.5 价格等价的混合令牌报告。我们将扩展按 GPT-5.5 计费，搜索时任务代理评估调用按 Nemotron 3 Ultra 计费。GPT-5.5 使用每百万输入 \$5 和输出 \$30/M [53]，Nemotron 3 Ultra 使用 DeepInfra 定价的每百万输入 \$0.5 和输出 \$2.2[61]。与图 9 观察到的成本拆分类似，约 20% 的搜索成本为扩展并按 GPT-5.5 价格计费，80% 为重复任务代理评估。重复评估调用以输入为主，因此主要使用保守的输入价格比 $0.5/5 = 0.10$ 进行核算；输出价格比较小（ $2.2/30 = 0.073$ ）。因此 Nemotron 3 Ultra 混合令牌总量 B 转换为约 $0.20B + 0.80(0.10B) = 0.28B$ GPT-5.5 价格等价混合令牌。这为选定的 Nemotron 3 Ultra 审稿人代理提供了 1.87M GPT-5.5 价格等价成本（ $B = 6.68M$ ）。

C.5 初始代理与提示

所有实验的初始工作区完全相同；每个任务智能体都从以下最小模板开始，外加一个简短的特定领域输出格式。元智能体指令明确指出，效用是在它从未见过的留出验证集上计算的，并且记忆的训练答案不会迁移到该验证集上，因此第 3.2 节中可见的训练反馈为自我修改提供信息，但并非选择目标。研究中唯一人工设计的提示属于对照运行：冻结的 Sakana 审稿人和 ProofAutoGrader。除这些格式（检查清单、评分标准、接受标准）之外报告的每个评估器行为都是由搜索产生的（附录 E）。

每个任务智能体都是来自共享模板的一次大语言模型调用，没有特定角色的代码。

Shared task-agent template (all roles)

```
You are an agent.

Task input:
{inputs}

{output_format}
```

特定领域的输出格式向该模板附加一个简短的架构。下面以论文审稿人为例；作者、IMO 评分员和证明者以及代码审稿人仅在他们请求的架构上有所不同，而编码员是初始配置中唯一具有工具访问权限的角色（bash、编辑器和模型委派；初始工作区每个任务最多允许 16 次工具调用，搜索后来在两条获胜链上都提高了这一预算，表 8）。无法解析的输出计为失败。

Initial output format: paper reviewer (also scores writer outputs and the adversarial pool)

```
You are reviewing the paper above for a top ML venue. Read it carefully
and decide whether to Accept or Reject it.

Respond with one JSON block:
{
  "Summary": "...", "Strengths": [...], "Weaknesses": [...],
  "Decision": "Accept" | "Reject"
}
For "Decision", use only "Accept" or "Reject".
```

自我修改提议者逐字接收以下指令；{repo_path}、{iterations_left}、和 {eval_path} 由测试框架填充。

The meta-agent instruction (verbatim, all headline runs)

```
Modify any part of the codebase at `{repo_path}` to improve performance
on the active evaluation domains. This includes your OWN code -- the
meta-agent's loop, prompts, tool surface, and the task-agent harness.
Improving your own self-improvement abilities compounds across future
generations and may lead to better outcomes in the long run than only
```

optimizing the per-domain task agents.

You have {iterations_left} expansions remaining.

Available LLM models for one-shot delegation are listed in `{repo_path}lineage/MODEL_CATALOG.md`. Use the `query_model` tool to fire a one-shot call to a different model with (prompt, id, effort, max_tokens) -- useful for offloading bulk text generation or sanity-checking your reasoning against a different model. The call has no chat history and no tools; it's tokens in, tokens out.

Live remaining budget for this expansion is prepended to every chat turn by the host harness. The host harness/proxy is the only budget authority; do not infer spend from local transport-library estimates, token estimates, or model-provider defaults. Read the current remaining budget before each action, including tool use and one-shot model delegation, and scale the number and size of calls to fit it.

Prior generation artifacts are at `{eval_path}`. Each ancestor `gen_id` contains:

- `<domain>_eval/predictions.csv` -- task-agent outputs paired with ground-truth labels on the TRAIN split. The only sample-level outcome data you can see directly.
- `agent_output/meta_agent_chat_history.md` -- the full transcript of that predecessor's reasoning. The highest-signal artifact: what was tried, what worked, what failed. Future generations will read YOUR chat history, so explain your reasoning clearly.
- `agent_output/model_patch.diff` -- the code edit the predecessor's meta-agent produced. Absent on `gen_initial/` (no agent edits there).
- `metadata.json` -- parent_genid, parent_agent_success, lineage info.

CRITICAL: your utility is computed on a HELD-OUT validation set you NEVER see. The train predictions you can read are feedback for study only. Do NOT hardcode answers, memorize specific question ids, or otherwise overfit to the train samples -- those tricks score perfectly on train and 0 on validation, which is what actually drives selection.

Outcomes are binary (1=correct, 0=incorrect) and a node's utility is the mean of its validation outcomes per (role, task). The evaluator is external -- improving your agents to genuinely produce better outputs is the path forward; gaming the parser or breaking the harness scores 0.

Each `<domain>_eval/report.json` lists `question_ids_passed` and `question_ids_failed`. A failed qid means either the LLM's output couldn't be parsed (counts as outcome=0 by convention) or it parsed but predicted the wrong answer; both are visible in `predictions.csv` row-by-row.

元智能体的运行时常量见表 4；每次扩展的租约在每轮前添加的实时预算头中公布，训练和验证调用携带各自的租约。

D 结果与消融

D.1 原始留出计数

表 5: 第 5.1 节和第 5.2 节背后选择时的原始留出计数。

Reviewer	Accuracy	Accept.
HGM-H specialist (node 51)	88	45
RQGM specialist (node 49)	84	40
RQGM generalist, pre-repl. (node 78)	84	42
RQGM adversarial, post-repl. (node 31)	80	32
DGM-H (published)	73	25
Sakana prompt	63	13

Reviewers: $n = 100$ held-out APReS papers.

Best-belief agent	Passes
RQGM specialist (node 104)	119
RQGM generalist (node 100)	119
HGM-H (node 95)	116

编码器: $n = 166$ 留出多语言任务, 评估匹配协议。

表 6: 表 2 背后每位评分者的证明分数。

Prover	ProofAutoGrader			HGM-H grader (node 45)			RQGM grader (node 22)		
	Mean	P@6	P@7	Mean	P@6	P@7	Mean	P@6	P@7
Static IMO25 prover	4.05	55.0	55.0	4.10	55.0	55.0	4.05	55.0	55.0
HGM-H prover	3.55	50.0	45.0	3.65	50.0	45.0	4.00	55.0	45.0
RQGM prover (generalist)	3.55	50.0	45.0	3.70	50.0	45.0	3.95	55.0	45.0
RQGM prover (specialist)	3.85	55.0	40.0	4.20	60.0	45.0	4.95	70.0	60.0

D.2 机制消融

下文提供的消融实验分别隔离了受控效用演化、替换、对抗池和选择性擦除等组件, 每个组件在论文评审运行中逐一关闭 (表 7)。所有机制和超参数保持不变; 仅底层模型更换为 Nemotron 3 Super。与 GPT-5.5 运行相比, Nemotron 3 Super 较小的上下文窗口导致频繁压缩。此外, 其替换决策部分反映可靠性修复而非评审逻辑 (附录 D.3.2)。

固定批评者达到饱和。关闭替换后, 作者最终获得 75/75 篇被接受的验证论文, 这使冻结批评者的接受率轻易超过人类接受率。

仅启用替换 (关闭对抗池) 可摆脱该平台期。最终锚定结果为 52/57; 作者在每次替换后重新设定基准, 而非使移动评判者饱和, 因此替换足以推动进展, 而对抗池则影响所选审稿人 (第 5.4 节)。

若不进行擦除, 效用将保持陈旧。退役评论家行仍保留在官方效用中; 该槽位通过一个 $A \rightarrow B \rightarrow A$ 被取代评论家的回报而振荡; 随着陈旧证据的积累, 评估者替换停止触发, 导致搜索反复探索相同节点。

对抗池仅在搜索时进行替换匹配。对抗池主要用于更好地划定人类生成文本与人工智能生成文本之间的决策边界, 因此它主要影响下游审稿人对作者生成文本的评估。

表 7：在评估者独立锚点上的机制消融。所有运行均使用 12,288 评估预算。审稿人锚点准确率在各运行间相当，而作者接受率（†），由各运行自身的评论家评分，则不然。单元格报告均值 $\pm$ 95% Jeffreys 区间。

运行	替换	池	擦除	作者验证准确率 (%) [†]	审稿人锚点准确率 (%)
Full mechanism	on	on	on	90.1 $\pm$ 3.5	95.0 $\pm$ 10.3
Replacement only	on	off	on	89.5 $\pm$ 13.7	91.2 $\pm$ 7.4
No erasure	on	on	off	90.0 $\pm$ 10.7	92.9 $\pm$ 10.8
Fixed critic	off	off	–	100.0 $\pm$ 1.7	78.2 $\pm$ 4.4

D.3 成本与进展的来源

我们现在展示实际搜索过程在计算分配和编辑方面的演变。

D.3.1 成本分解

每次模型调用都与工作区扩展、训练时评估或验证评估相关。总体运行成本主要由验证主导：在三次主实验运行中，验证约占混合令牌的 65 – 69%，扩展占 18 – 23%，训练时评估（每个节点三个训练样本）占 12 – 14%（图 9）。为每个元代理分配更多训练样本可能因上下文更丰富而实现更高效的搜索，因此当前用于训练时评估的令牌比例可能过低。

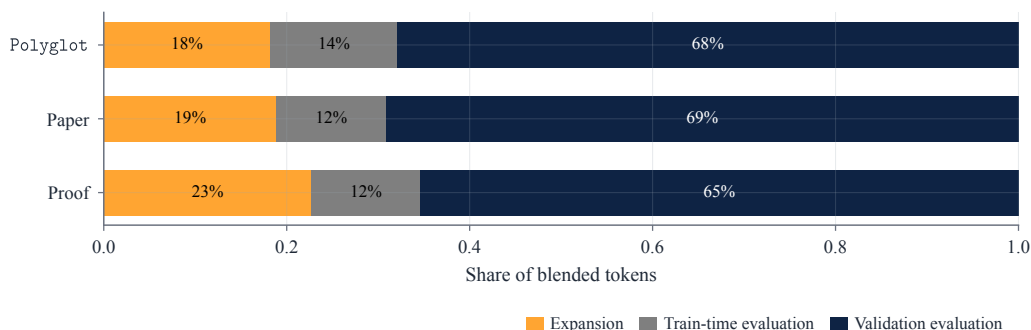


图 9：三次主要 GPT-5.5（低）运行的混合令牌成本分解，分为工作区扩展、训练时评估和验证评估。

D.3.2 补丁表面

我们根据每个谱系边补丁所修改的代码表面对其进行分类。共享表面，即多个角色共用的任务代理代码和基础设施，在每次 GPT-5.5 运行中承载了 59 – 90% 的被接受补丁（图 10）。例如，Polyglot 运行将其改进集中在跨角色泛化的共享和基础设施表面上。我们认为这是因为元代理意识到角色特定补丁各自驱动单一效用且泛化性较差。Nemotron 3 Super 运行是例外，它大量针对元代理的指令模块（占边的 42%），可能是为了提高其可靠性。

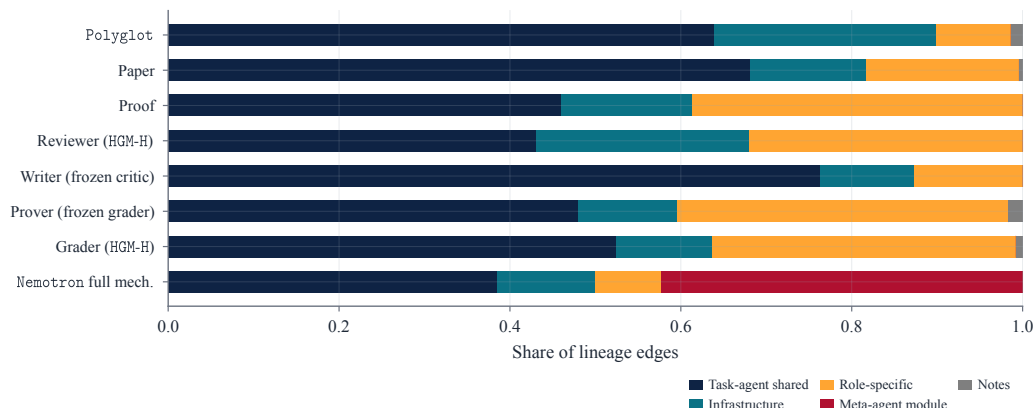


图 10：按运行划分的补丁表面，将每条被接受的谱系边按其修改的代码表面分类（任务代理共享、基础设施、角色特定、元代理模块或注释）。

E 共同进化发现了什么

每个评估者角色都是共享工作区文件中的一个提示条目，因此每个评估者的行为历史可直接从其谱系中读取。初始角色提示是最小化的（附录 C.5）；在所有角色中，搜索最常见的操作是将模糊的单行指令改写为具体、带标准的评分细则，其产生的规则引用如下。由于起始提示是最小化的，更强的初始提示会同等提高所有方法的得分，而不会改变共同进化比较，因为该比较使每种方法都从相同的起点出发。

E.1 进化后的评估者改变了什么

对抗性审稿人的谱系将一行初始指令改写为一次合取式接受标准，而替换前的通才则带有双向校准。

Evolved rule: the adversarial reviewer's conjunctive standard (gen 4)

... Use a realistic selective-conference standard: accept only if the paper has a clear nontrivial contribution, technically sound methodology/theory or experiments, adequate positioning against prior work, and claims supported by evidence; reject papers with missing validation, unclear novelty, serious technical gaps, or overclaiming. Do not be swayed by fluent writing alone.

Evolved rule: the displaced generalist's two-sided calibration

... Calibrate the decision holistically rather than requiring perfection: a paper with a coherent contribution and adequate evidence should be accepted despite presentation flaws, while impressive writing without substantiated experiments or technical substance should be rejected.

两种措辞在其谱系早期均已出现：对抗性池改变了被选中的谱系，以拒绝更大比例的 AI 生成论文。

E.2 共同进化的评分器

评分器仅通过两次补丁达到其报告形式，两次补丁均在此引用：初始指令在无评分细则的情况下分配四个标签之一。第一次补丁引入了角色设定、两步里程碑程序、分数映射和严格性规则，同一补丁还引入了证明者提示，因此证明者和评分器一同进入工作区。后续补丁移除了全面严格性，同时保留了里程碑程序和分数映射。

Evolved rule: the grader's first rewrite (gen 2)

You are an expert, conservative IMO proof grader. ... First identify the key required steps from the guidelines; then check whether the student's text actually proves them rigorously, ignoring unsupported claims, handwaving, and statements merely resembling the reference.

Use this mapping unless the problem-specific guidelines say otherwise:

- correct: a complete rigorous solution (7/7).
- almost: essentially complete with only a minor local gap or arithmetic slip (6/7); do NOT use for missing major lemmas.
- partial: substantial guideline-listed progress, but not a near-complete proof (1/7).
- incorrect: no substantial credited progress or a fundamentally flawed argument (0/7).

Be especially strict with advanced geometry/combinatorics solutions that invoke large unproved lemmas ...

Evolved rule: the co-evolved grader's recalibration (gen 22)

Calibrate generously but not naively: do not require the student's proof to match the reference solution, and do not downgrade merely because a standard theorem/lemma is cited tersely, the exposition is compressed, or a repairable statement is slightly misphrased. ... Downgrade to partial/incorrect only for a genuinely fatal gap, a false central claim with no reasonable repair, or failure to prove a guideline milestone.

Its behavior in the cross-grader panel of [Tab. 2](#) follows from these patches.

E.3 多语言代码审稿人

代码审稿槽位被替换了三次；初始指令要求对总体代码质量给出通过/不通过判定，三位在任者的规则引用如下。第三位从比第二位更早的祖先分支而来，因此它不继承第二位的要点。在三次替换中，该槽位逐渐转向证据受限的审稿：每位在任者都将失败的定义收窄至差异本身中的缺陷。

Evolved rules: the three promoted code reviewers

Node 19 (gen 5): You are a pragmatic senior maintainer reviewing a proposed patch. ...

- PASS when the patch plausibly fixes/adds the intended behavior, is localized, and has no clear correctness, compilation, security, or data-loss defect. ...
- FAIL only for a concrete blocker: obvious wrong behavior, broken API usage, compile/type errors visible in the diff, missing essential cases, flaky/meaningless tests, ...
- ... If evidence is balanced, prefer pass.

Node 34 (gen 34, appends): - Review in this order: (1) infer the intended user-visible behavior ..., (2) check whether production code implements it, (3) look for definite regressions or syntax/type errors, (4) only then consider style, test coverage, and maintainability. Style concerns without a concrete correctness risk should not cause fail. ... FAIL only when you can point to a specific scenario that remains broken ...

Node 86 (gen 59, extends the blocker rule): ... Do not invent blockers from missing project context; the diff alone must show the defect.

- PASS broad but coherent mechanical migrations, CI/build-artifact pipeline updates, ... and test-only fixes when they are internally consistent ...
- For security/credential-looking files, fail only for an actual exposed secret or dangerous permission; ...

E.4 获胜谱系

我们现在展示所有运行中的完整谱系。每个存档节点存储其父节点、作为统一差异的累积自我修改，以及产生它的元代理记录；图 11 绘制了三个存档，表 8 至表 10 列出了每一步，引用的短语来自元代理自己的总结。通才链的最大收益不是提示更改，而是代码修复，并且同一节点在第一个检查点被提升到代码审查槽位。对抗性审稿人的标准在任何替换之前就已出现；对抗性池在第二个训练轮次选择了该谱系（附录 E）。

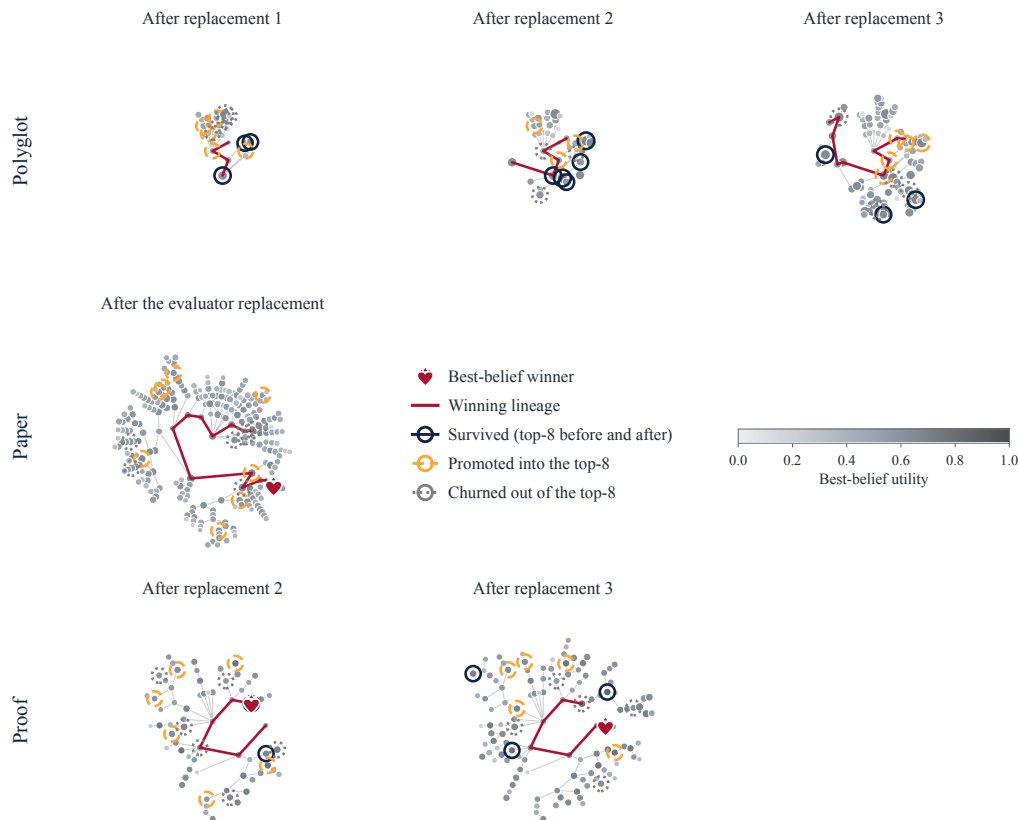


图 11：在三个主要领域中，每个评估器替换保留了什么与重新排序了什么。行是领域（从上到下：多语言、论文、证明）；列是该运行中显示的评估器替换，每个替换绘制了当时存档的状态（径向布局，节点颜色是最佳信念效用，大小是证据计数）。在每个替换的前 8 名排行榜节点中，深蓝色环在重新排序后幸存，琥珀色环是新标准提升到前 8 名的谱系，灰色环是被淘汰的。排名节点少于 20 个的替换被省略，如图 3 所示，因此证明运行的第一次替换未显示。

后来在第一个检查点被提升到代码审查槽位。对抗性审稿人的标准在任何替换之前就已出现；对抗性池在第二个训练轮次选择了该谱系（附录 E）。

评估器提示之外的两项修改最为重要。第一项是节点 19 的差异卫生修复（清单 1）；工件过滤器是承重行，引用自其补丁。

清单 1：节点 19 的差异卫生修复（多语言运行）：未跟踪的依赖工件和不安全的符号链接从提交的补丁中被过滤掉。

```
diff --git a/utls/git_utils.py b/utls/git_utils.py @@ def diff_versus_commit(git_dname, commit):
dependency_artifact_roots = { "node_模块", ".npm", ".yarn", ".pnpm-store", "target", "build", "dist",
"__pycache__", ".pytest_cache", } def should_include_untracked(relpath): parts = relpath.split(os.sep) 如果
parts 且 parts[0] in dependency_artifact_roots: return False # 跳过偶然的安装工件 file_path =
os.path.abspath(os.path.join(git_dname, relpath)) 如果 os.path.islink(file_path): target =
os.path.realpath(file_path) # 丢弃解析到检出目录之外的符号链接 如果非
target.startswith(os.path.abspath(git_dname)): 返回 False 返回 True
```

表 8：多语言获胜链；两个最终最佳信念代理在保留数据上得分 119/166。

Node	Modification
<i>specialist</i> (final 119/166)	
4	coder tool budget 16→40; review calibration; catalog-path fix
35	coder workflow: one tool call at a time, prefer exact algorithms
51	review balance against blindly passing bad patches
104	parse retry; label normalization; concrete-blocker review rule
<i>generalist</i> (final 119/166)	
5	parse retry; tool budget 16→24; concrete-blocker review
19	untracked-artifact and symlink diff hygiene
33	the same hygiene applied to a second code path
42	one tool call per turn; preserve existing public APIs
72	read problem text and tests first; wider artifact filter
78	final-output reminder; conservative label normalization
93	dedicated artifact-review format; pass plausible localized fixes
100	JSON-serialized tool results (transcript-corruption fix)

表 9：论文领域获胜链。作者链经过报告的审稿人（49）和通才（78）；对抗性审稿人（31）单独下降。

Node	Modification
<i>writer</i>	
7	stronger authoring prompt; runtime prompt nudges
11	parse-repair retry; 8,192-token writer output cap
17	present raw paper text instead of the input dictionary
37	remove global LLM monkeypatch
49	writer returns raw text, not JSON-wrapped continuations
78	two-sided review calibration; no acceptance-seeking prose
81	format hygiene in the client; revert import hook that could hang
89	continue from the exact stopping point; longer word target
150	liberal unwrapping of writer output (recovers wrapped papers)
154	area-chair review standard ; bounded-evidence writing rules

对抗性 4 合取选择性会议评审标准 25 作者侧评审视为可行性
 门槛 31 自包含修订论文；4,096 词元输出上限

第二项是证明者对其祖先自修订功能的三行撤回声明（节点 66，清单 2）。

清单 2：节点 66 的撤回声明（证明运行）：继承的自修订通道默认禁用，修订指令被改写为保留可靠草稿，记录的理由是修订“可能将正确或接近正确的详细证明退化为依赖未证明的冷门命名定理的较弱黑盒引用式证明。”

```
diff --git a/task_agent.py b/task_agent.py
@@ class TaskAgent (AgentSystem):
-     IMO_PROOF_REVISION_ENABLED = os.environ.get("HYPERAGENTS_IMO_PROOF_REVISION",
-         ↪ "1") != "0"
+     IMO_PROOF_REVISION_ENABLED = os.environ.get("HYPERAGENTS_IMO_PROOF_REVISION",
+         ↪ "0") == "1"
@@ Draft proof:
- 找出任何隐藏的漏洞……如果草稿可靠，保留但使其更清晰。+ 如果草稿本质上可靠则保留；不要用依赖未证明的
冷门命名定理的较短答案替换详细证明。
```

表 10: 证明域获胜链。节点 22 处评分器的校准逆转是任何获胜链中最大的单步效用增益。

Node	Modification
<i>grader</i>	
2	conservative rubric-aware grading; first prover prompt
22	calibration reversal: do not downgrade terse or compressed proofs
<i>prover</i>	
3	parse retry; lemma-justification and case-coverage prompt
11	partial-versus-incorrect distinction; hidden-assumption checks
19	second-pass self-revision with an adversarial checker
66	disable the inherited self-revision (rollout below)

F 理论

F.1 设定: HGM 搜索算法

RQGM 构建于赫胥黎-哥德尔机 (HGM) [3] 之上, 该机器将自我改进形式化为由分支元生产力 (CMP) 引导的树搜索, 以及超智能体 (DGM-H) [4], 后者在可编辑智能体工作空间中引入元认知自我修改; 本文仅回顾所需的假设和理论, 然后陈述证明所使用的局部运行条件。

F.1.1 自我改进作为树搜索

遵循 Wang 等人 [3] 的方法, 一个智能体档案 $\mathcal{T}_t$ 构成一棵树, 初始化为 $\mathcal{T}_0 = \{a_0\}$ 。在每次迭代中, 策略要么修改现有智能体, 要么评估一个智能体:

$$\mathcal{A}_t = \mathcal{M}_t \cup \mathcal{V}_t, \\ \mathcal{M}_t = \{m_a : a \in \mathcal{T}_t\}, \quad \mathcal{V}_t = \{v_a : a \in \mathcal{T}_t\}.$$

修改动作 m_a 产生 a 的子节点; 评估动作 v_a 测试 a 在一个任务并返回二元结果 $o \in \{0, 1\}$ 。在固定的外部预算 B 耗尽后, 系统返回档案中得分最高的智能体: $a_{\text{最终}} = \arg \max_{a \in \mathcal{T}_g} \text{得分}(\pi(a))$ 。

效用 $U(a)$ 是智能体 a 在评估协议下的期望二元成功率。RQGM 保留了这种扁平的 HGM 搜索算法, 同时推广了节点包含的内容及其二元结果的产生方式。

F.1.2 HGM 兼容的操作条件

These are the local conditions the appendix results need.

假设 1 (HGM 兼容的固定标准搜索)。对于固定标准自我改进轮次的理论分析, 假设: **C1.** 目标通过搜索预算耗尽后从档案中选出的智能体进行评估; 中间观察指导搜索但不作为单独奖励。 **C2.** 对于活跃标准下的任何固定节点-任务对, 重复评估产生时间齐次的二元结果。 **C3.** 理想 HGM 定理使用的任何预言机或证明对象均在经验搜索评估预算之外。 **C4.** 经验调度器通过运行使用的外部固定预算单位来考虑扩展和评估。

RQGM 的关键条件是平稳性: HGM 假设评估器固定, 因此早期结果与晚期结果含义相同。RQGM 在局部保留该性质: 第 3.4 节在一个轮次内冻结学习到的评估器, 第 3.5 节仅允许在检查点进行更改, 随后选择性地擦除其含义依赖于被替换评估器的记录。

F.1.3 分支级元生产力

Wang 等人 [3] 表明, 节点的即时基准性能可能无法预测长期的自我改进价值: 一个弱节点可能产生强后代。HGM 用对观察到的后代生产力的档案搜索取代了哥德尔机对未来修改的不可行证明搜索, 根据节点的分支 (即其后代的子树 $C(\mathcal{T}, a)$ of a and) 对节点进行评分, 当树结构清晰时记为 $C(a)$ 。

定理 1 (Wang 等人 [3])。在假设 1 下, 访问 CMP 预言机足以实现哥德尔机。

在实践中, HGM 通过汇总分支上的二元结果来估计分支级元生产力:

$$\begin{aligned}\widehat{\text{CMP}}(a) &= \frac{n_{\text{success}}^C(a)}{n_{\text{success}}^C(a) + n_{\text{failure}}^C(a)}, \\ n_{\text{success}}^C(a) &= \sum_{a' \in C(a)} n_{\text{success}}(a'), \\ n_{\text{failure}}^C(a) &= \sum_{a' \in C(a)} n_{\text{failure}}(a').\end{aligned}$$

对分支级成功与失败计数进行汤普森采样, 在探索与利用之间进行权衡, UCB-Air 门 [49] 控制树的生长。RQGM 原封不动地继承了这些机制; 证明所依赖的性质是, 调度器消耗每个节点的扁平二元结果流, 并在节点的分支上进行聚合。节点表示是 HyperAgents [4] 的元认知工作区: 每个档案节点是一个共享工作区 w_a with K 角色代理加上一个可以编辑节点代码和协调的元代理, 而 HGM 搜索决定哪个谱系获得扩展和评估预算 (附录 C.2)。

F.2 替换与选择的一条规则

评估器槽位在一个训练轮次内冻结, 挑战者根据锚定证据上的 ϵ -最佳信念分数进行选择, 被提升的评估器在下一个训练轮次中冻结 (第 3.5 节)。

F.2.1 评估器替换规则 在检查点 c 处, 固定一个评估器槽位 m 。令 $e_m^{(j_m)}$ 为现任冻结评估器, 令 $C_m(c)$ 为在 GT_m 上具有所需评估器独立锚点证据的挑战评估器。定义

$$\mathcal{E}_m(c) = \{e_m^{(j_m)}\} \cup C_m(c).$$

For each candidate $e \in \mathcal{E}_m(c)$, let S_e^{gt} 及 b_e 为其在锚点上的成功/失败。计算

$$BB_\epsilon(e) = I_\epsilon^{-1}\left(1 + S_e^{\text{gt}}, 1 + F_e^{\text{gt}}\right),$$

与最终智能体选择 (第 3.1 节) 所用相同的 ϵ -最佳信念分数。由于 $BB_\epsilon(e)$ 是候选者在锚点结果上的贝塔后验的 ϵ -分位数, 因此它是候选者真实准确率的保守估计, 候选者以概率 $1 - \epsilon$ 超过该估计。所选评估器为 $e_m^* \in \arg \max_{e \in \mathcal{E}_m(c)} BB_\epsilon(e)$, 平局时偏向现任者; 若 $e_m^* = e_m^{(j_m)}$

, no transition occurs.

F.3 工作后验一致性

固定一个轮次向量 j 和一个节点 a 。令 $C_j(a) = \{(r, d) : r \in \mathcal{R}^{\text{elig}}(a, j), d \in \mathcal{D}_{r, \text{elig}}(a, j)\}$ 表示节点 a during 轮次 j 的合格角色-任务单元的有限集合。对于单元 $c = (r, d) \in C_j(a)$, 令 $p_{j,c}(a)$ 为在轮次 j 的活跃评估标准下, 节点 a on 角色 r 和任务 d 的一次搜索评估的固定成功概率。默认的角色/任务平衡目标分配为

$$w_{r,d} = \frac{1}{|\mathcal{R}_{\text{elig}}(a, j)|} \cdot \frac{1}{|\mathcal{D}_{r, \text{elig}}(a, j)|},$$

对于较小的合格集合进行明显的重新归一化。一般地, 令 $w_c \geq 0$ 为满足以下条件的权重

$$w_c = 1, \quad c \in C_j(a)$$

其中 $w_c \geq 0$ (见命题 3)。相应的轮次局部角色 - 任务平衡效用为

$$U_j(a) = \sum_{c \in C_j(a)} w_c p_{c,j}(a).$$

命题 1 (均衡角色-任务采样下 HGM 工作后验的一致性)。 固定一个训练轮次向量 $\mathbf{j}$ 、一个节点 a 以及合格的角色-任务单元集 $C_j(a)$ 。考虑该训练轮次中节点 a 被选中的搜索评估子序列。经过 n 次此类评估后, 令 $n_c(n)$ 为分配给单元 c 的评估次数, 并令

$$S_a(n) = \sum_{i=1}^n O_i$$

为节点 a 记录到的二元成功总数。假设节点内调度器是 w -均衡的, 即:

$$\frac{n_c(n)}{n} \rightarrow w_c \quad \text{for every } c \in C_j(a).$$

同时假设, 对于每个单元 c , 对 c 上 a 的重复评估满足单元级强大数定律:

$$\frac{1}{m} \sum_{i=1}^m O_{c,i} \rightarrow p_{c,j}(a) \quad \text{a.s. as } m \rightarrow \infty,$$

其中 $O_{c,i}$ 是从单元 c 观测到的第 i 个结果。例如, 当单元级结果是具有固定成功概率 $p_{c,j}(a)$ 的独立伯努利试验时, 或在任何满足强大数定律的平稳鞅差模型下, 该条件成立。定义 HGM 兼容的工作后验

$$Q_n(a) = \text{Beta}(1 + S_a(n), 1 + n - S_a(n)).$$

Its mean is

$$M_n(a) = \frac{1 + S_a(n)}{2 + n}.$$

Then

$$M_n(a) \rightarrow U_j(a) = \sum_{c \in C_j(a)} w_c p_{c,j}(a) \quad \text{a.s.}$$

因此, 合并的贝塔累加器在渐近意义下保持了正确的角色-任务均衡后验均值目标。 $Q_n(a)$ 是用于保留汤普森采样平坦 HGM 成功/失败接口的工作后验。

证明。固定训练轮次向量 $\mathbf{j}$ 、节点 a 以及合格单元集 $C_j(a)$ 。将 p_c 记为 $p_{c,j}(a)$ 。令 $n_c(n)$ 为该训练轮次中节点 a 的前 n 次搜索评估中单元 c 被评估的次数。令

$$\hat{p}_{c,n} = \frac{1}{n_c(n)} \sum_{i=1}^{n_c(n)} O_{c,i}$$

当 $n_c(n) > 0$ 时。具有 $w_c = 0$ 的单元贡献的评估比例趋于零, 并从极限中消失, 因此单元级强大数定律仅应用于支撑集 $\{c : w_c > 0\}$ 上。对于每个这样的单元, $w_c > 0$ 和 $n_c(n)/n \rightarrow w_c$ 给出 $n_c(n) \rightarrow \infty$, 因此根据假定的单元级强大数定律, $\hat{p}_{c,n} \rightarrow p_c$ 几乎必然成立。

The pooled empirical success rate decomposes by cells:

$$\frac{S_a(n)}{n} = \sum_{c \in C_j(a)} \frac{n_c(n)}{n} \hat{p}_{c,n}.$$

取极限并利用 $C_j(a)$ 的有限性,

$$\frac{S_a(n)}{n} \longrightarrow_{c \in C_j(a)} w_c p_c = U_j(a) \quad \text{a.s.}$$

分层高斯混合模型的工作后验为

$$Q_n(a) = \text{Beta}(1 + S_a(n), 1 + n - S_a(n)),$$

with mean

$$M_n(a) = \frac{1 + S_a(n)}{2 + n}.$$

Moreover,

$$\left| \frac{1 + S_a(n)}{2 + n} - \frac{S_a(n)}{n} \right| = \left| \frac{n - 2S_a(n)}{n(n+2)} \right| \leq \frac{1}{n+2}.$$

因此

$$M_n(a) - \frac{S_a(n)}{n} \longrightarrow 0,$$

且由此

$$M_n(a) \longrightarrow U_j(a) \quad \text{a.s.}$$

关于后验解释: 单元分层似然 $\pi_c \in C_{j(a)} p_c^{S_c(n)} (1-p_c)^{n_c(n)-S_c(n)}$, 其中 $S_c(n)$ 为单元 c 中的成功次数, 仅当所有 p_c 相等时才是单一伯努利参数的后验。因此 $Q_n(a)$ 并非角色-任务混合的精确贝叶斯后验, 而是一个工作后验, 它保留了平坦的分层高斯混合模型累加器 ($S_a(n)$, $n - S_a(n)$), 同时对 $U_j(a)$ 保持均值一致。合并的工作后验相对于平衡分层估计量过度离散, 因此对下文的保守下界解读而言是保守的。□

F.4 轮次局部固定效用有效性

槽位局部效用准则。对于每个评估器槽位 $m \in \{1, \dots, M\}$, 令 $\kappa_m^{(j_m)}$ 表示在轮次索引 j_m 处生效的槽位局部效用准则。槽位准则包括所有可能影响依赖该槽位的记录的效用证据分布的槽位局部对象。它不包括派生搜索统计量, 这些统计量在转换后根据保留的效用证据重新计算。令

$$\kappa^j = (\kappa_1^{(j_1)}, \dots, \kappa_M^{(j_M)})$$

denote the active criterion vector at epoch vector $j = (j_1, \dots, j_M)$.

假设 2 (训练轮次内平稳的学习评估)。固定一个训练轮次向量 j 。对于每个依赖评估器的节点-角色-任务单元 (a, r, d) , 以下对象在整个训练轮次内保持不变: (i) κ^j 中影响该单元的所有槽位准则; (ii) 节点 a 使用的节点工作区; (iii) 任务 d 的工件生成、重放或对抗池采样协议; 以及 (iv) 最终二元评分规则。等价地, 若 X 表示提交给二元评分器的对象, 则在该训练轮次内

$$X \sim P_{a,r,d}^j, \quad O \sim K_{r,d}^j(\cdot | X), \quad O \in \{0, 1\},$$

其中工件分布 $P_{a,r,d}^j$ 和评估器核函数 $K_{r,d}^j$ 在该训练轮次内时间齐次, 且不受该训练轮次中先前搜索结果的影响。确定性评估器情形通过允许 $K_{r,d}^j(\frac{1}{\square} | x) \in \{0, 1\}$ 予以涵盖。

该假设仅断言训练轮次局部平稳性: 在重复调用间独立抽取的情况下, 结果服从具有固定参数的伯努利分布; 而在无独立性的情形下, 下文论证仅使用固定的训练轮次局部条件均值。

效用证据与缓存工件。效用证据记录不同于原始工件或审计日志: 工件、谱系报告和不可变审计记录可在效用转换间保留,

但它们仅通过符合准则的记录对节点级或分支级效用统计作出贡献。重放或重新评分的工件会为实际使用的任务生成一条新记录，并标记当前准则。

效用证据记录写作

$$z = (a(z), r(z), d(z), o(z), \text{dep}(z), \kappa(z), j(z)),$$

其中 $a(z)$ 为节点， $r(z)$ 为角色， $d(z)$ 为任务， $o(z) \in \{0, 1\}$ 为二元结果。集合 $\text{dep}(z) \subseteq \{1, \dots, M\}$

包含每个评估器槽位，其准则影响了记录，无论是通过工件生成、任务或对抗池采样、重放分布，还是最终二元评分。标签 $\kappa(z)$ 存储记录生成时依赖槽位上的活跃准则标签， $j(z)$ 存储生成时的

轮次向量。评估器无关记录的依赖 $\text{dep}(z) = \emptyset$ 。

定义 F.1 (准则有效记录)。效用证据记录 z 在活跃准则向量 κ^j 下有效，当且仅当对于每个 $\ell \in \text{依赖}(z)$ ， $\kappa_\ell(z) = \kappa_\ell^{(j_\ell)}$ 。依赖 $\text{dep}(z) = \emptyset$ 的记录在每个评估器轮次下均有效。树档案 T 在 κ^j 下准则一致，如果 T 中每个保留的效用证据记录在 κ^j 下均有效。

定义 F.2 (槽位 m 上的效用转换)。评估器槽位 m 上的效用转换通过将 $\kappa_m^{(j_m)}$ 替换为 $\kappa_m^{(j_m+1)}$ ，推进 $j_m \rightarrow j_m + 1$ 。新准则是该槽位的下一个冻结评估器，以及轮次开始前选定的任何固定重放或验证分布。它必须在新效用证据收集之前固定。效用转换随后通过应用擦除 m 过滤过时效用证据。对于每个节点 a ，令 $\mathcal{Z}_a$ 表示其保留的效用证据记录。转换到轮次向量 j' 后，其中 j' 与 j 仅在坐标 m 上不同，定义

$$\text{Erase}_m(\mathcal{Z}_a) = \left\{ z \in \mathcal{Z}_a : m \notin \text{dep}(z) \text{ or } \kappa_m(z) = \kappa_m^{(j'_m)} \right\}.$$

对每个节点应用擦除 m 后，所有派生的充分统计量均从保留的效用证据记录重新计算。这些派生量包括节点级成功与失败计数 S_a 、失败 $\bar{a}$ 、角色与任务计数器 n_r (角色) 和 n_d (角色)、分支级成功与失败计数、汤普森采样统计量、槽位局部验证计数器以及缓存的分数摘要。

原始工件缓存和不可变审计日志可保留在档案中，但除非它们产生新的准则有效效用证据记录，否则不计入这些统计量。

命题 2 (选择性擦除保持准则一致性)。考虑在槽位 m 上从轮次向量 j 到轮次向量 j' 的效用转移，其中 j' 与 j 仅在坐标 m 处不同。假设档案 T 在转移前在 κ^j 下是准则一致的。在将 $\kappa_m^{(j_m)}$ 替换为 $\kappa_m^{(j'_m)}$ 、应用擦除 m 并根据保留的效用证据记录重新计算所有派生统计量之后，档案在 $\kappa^{j'}$ 下是准则一致的。此外，在新轮次期间生成并标记为活动依赖准则的任何后续效用证据记录在 $\kappa^{j'}$ 下都是有效的。

证明。设 z 为擦除 m (定义 F.2) 后保留的任何记录，并固定 $\ell \in \text{依赖}(z)$ 。如果 $\ell = m$ ，保留强制擦除 m ，所以 $\kappa_m(z) = \kappa_m^{(j'_m)}$ 的第二个子句。如果 $\ell \neq m$ ，转移使槽位 ℓ 保持不变 ($\kappa_\ell^{(j_\ell)} = \kappa_\ell^{(j'_\ell)}$)，并且先前的准则一致性给出 $\kappa_\ell(z) = \kappa_\ell^{(j_\ell)} = \kappa_\ell^{(j'_\ell)}$ 。因此 z 在 $\kappa^{j'}$ (定义 F.1) 下是有效的。由于所有派生统计量都完全根据保留的记录重新计算 (定义 F.2)，没有过期记录进入其中，并且之后生成的任何记录都标记为活动依赖准则，因此根据定义是有效的。因此档案在 $\kappa^{j'}$ 下保持准则一致，直到下一次转移。□

备注 1（不相交槽位上的转移可交换）。对于 $m \neq \ell$ ，擦除 m 和擦除 ℓ 是记录集上的逐点过滤器：每个都恰好移除其依赖集包含其被替换槽位的记录，并且都不修改保留的记录。以任一顺序应用两者，都会移除两个受影响记录集的并集并保留相同的档案；由于准则替换作用于 κ 的不同坐标，并且所有派生统计量都根据保留的记录重新计算，转移后的状态不依赖于处理不相交槽位上检查点的顺序。

备注 2（评估器相关的验证记录，包括对抗池，被擦除）。一些槽位局部验证记录是为了暴露前任评估器的盲点而生成的，其中第 5.4 节的对抗池就是其中之一。此类记录的结果取决于对其进行评分的评估器，因此其依赖集包含该槽位，并且当该评估器被替换时，擦除 m 会将其移除。因此这些记录在产生它们的轮次内影响节点效用，并且从不参与评估器替换，评估器替换选择的是与评估器无关的锚点 GT_m 。

接下来我们陈述固定训练轮次有效性结果，这是一个接口定理：在固定一个训练轮次向量和一个准则一致的档案后，该训练轮次中保留的效用证据和后续评估指向一个与分层高斯混合搜索兼容的固定二元结果问题。

命题 3（训练轮次局部固定准则有效性）。固定一个训练轮次向量 j ，并假设档案在活跃准则向量 κ^j 下是准则一致的。假设所有依赖评估器的角色在其活跃冻结准则下满足假设 2，且所有不依赖评估器的角色满足假设 1 中的分层高斯混合评估条件。那么，在这个固定训练轮次内，每个合格的节点-角色-任务元组 (a, r, d) 都诱导出一个具有固定成功概率的时间齐次二元结果律。

$$p_{r,d,j}(a) = \Pr_{\kappa^j}(O = 1 \mid a, r, d).$$

因此，该训练轮次定义了一个固定准则的二元结果搜索问题，其训练轮次局部效用为 $U_j(a) = \sum_{(r,d) \in C(a)} w_{r,d}^j p_{r,d,j}(a)$ ，其中 $C_j(a)$ 是节点 a 在该训练轮次中的合格角色-任务单元集， $w_{r,d}^j \geq 0$ 是固定的训练轮次局部目标权重，并在 $C_j(a)$ 上归一化。因此，任何以固定二元结果准则和访问相应精确条件均值预测预言机为假设的分层高斯混合预言机级定理，都适用于 U_j 下相应的理想化训练轮次局部预言机问题。经验汤普森采样实现使用保留的二元记录作为该固定训练轮次问题的工作估计器；它本身并不是精确条件均值预测预言机的实现。

证明。固定一个训练轮次向量 j ；活跃准则向量 κ^j 则按定义固定。我们证明每个合格的节点-角色-任务元组 (a, r, d) 具有固定的二元结果分布。

如果角色 r 不依赖评估器，假设 1 中的分层高斯混合评估条件意味着对 (a, r, d) 的重复评估产生的二元结果，其分布以及期望值不随评估时间或先前搜索事件而改变，因此存在一个固定的成功概率 $p_{r,d,j}(a) = \Pr(O = 1 \mid a, r, d)$ 。

若角色 r 依赖于评估器，则令 X 表示提交给二元评分器的对象。该对象可以是新生成的工件、通过固定反馈流水线生成的修订工件，或从冻结回放池或对抗池分布中采样的项。根据假设 2，其分布 $P_{a,r,d}^j$ 在该训练轮次内固定。二元评分规则也是固定的：在 $X = x$ 条件下，结果要么是确定性的，要么从固定评估器核函数 $K_{r,d}^j(\cdot \mid x)$ 中采样。因此

marginal success probability $p_{r,d,j}(a) = \Pr_{\kappa^j}(O = 1 \mid a, r, d) = \int K_{r,d}^j(1 \mid x) dP_{a,r,d}^j(x)$ is fixed 在整个训练轮次中，确定性情形 $K_{r,d}^j(1 \mid x) \in \{0, 1\}$

因此，每个符合条件的元组 (a, r, d) 都会以固定的轮次局部成功概率诱导二元结果。由于档案在 κ^j 下满足准则一致性，搜索过程使用的每条保留效用证据记录都是在所有影响该记录的槽位上与活跃准则匹配的准则标签下生成的。

根据构造，同一训练轮次内后续生成的记录都带有相同的活跃依赖准则标签。

轮次局部效用

$$U_j(a) = \sum_{(r,d) \in C_j(a)} w_{r,d}^j p_{r,d,j}(a)$$

因此是 a 对训练轮次持续时间的固定函数。HGM 搜索算法要求每个被评估节点产生二元结果流，并在固定效用准则下对节点和分支进行聚合。该要求在该训练轮次内得到满足，因为每次搜索评估都产生 $O \in \{0, 1\}$ ，且准则一致性不变量确保用于节点级和分支级统计的记录在当前轮次准则下有效。

由此可知，在固定轮次向量 j 、活跃准则向量 κ^j 以及准则一致性档案的条件下，当前训练轮次定义了一个 HGM 形式的普通固定准则二元结果搜索问题。因此，其假设由该理想化固定轮次问题满足的 HGM 预言机级定理在 U_j 下适用于该问题。□

F.5 受控效用演化的支撑性结果

真实最佳信念替换。 回顾槽位 m 替换规则及记号 $\mathcal{E}_m(c)$, $\tilde{e}$, 以及附录 F.2.1 中的 S_e^{gt} ,

$BB_e(e) = I_e^{-1}(1 + S_e^{\text{gt}}, 1 + F_e^{\text{gt}})$, 其中检查点规则在 $\arg \max_{e \in E^{m(c)}} BB_e(e)$ 中选择一
 个元素，并偏向现任者。
 注 3 (仅锚点评估器替换)。得分 $BB_e(e) = I_e^{-1}(1 + S_e^{\text{gt}}, 1 + F_e^{\text{gt}})$ 仅依赖于与评估器无关的锚点计数 ($S_e^{\text{gt}}, F_e^{\text{gt}}$) (附录 F.2.1)；没有槽位相关的效用记录进入其中。因此，所选的下一个评估器仅是现任者、挑战者集合以及这些锚点计数的函数，并且由于现任者位于候选集中且平时偏向它，除非挑战者严格最大化得分，否则不会发生转移。因此，在检查点处的共同锚点证据上，被提升的评估器的锚点最佳信念至少不低于现任者的：替换在做出时绝不比保留更差。

命题 4 (最佳信念下界轨迹)。固定一个训练轮次和一个有限候选集 A 。对于每个候选 $a \in A$ ，令其轮次局部效用的 HGM 工作后验为

$$Q_a = \text{Beta}(1 + S_a, 1 + F_a),$$

and define

$$BB_e(a) = I_e^{-1}(1 + S_a, 1 + F_a).$$

If $a^* \in \arg \max_{a \in A} BB_e(a)$, then, under the working posterior for a^* ,

$$\Pr(U(a^*) \geq BB_e(a^*) \mid Q_{a^*}) = 1 - \epsilon.$$

更一般地，对于任意序列的 L 报告选择 $a_1^*, \dots, a_L^*$ ，如果相应的工作后验是校准的，那么以至少 $1 - L\epsilon$ 的概率，每个被选候选的效用同时超过其报告的最佳信念下界。

证明。根据定义 $BB_e(a)$ 是 Beta 工作后验 Q_a 的 ϵ 分位数 (附录 F.2.1)，因此 $\Pr_{U \sim Q_a}(U \geq BB_e(a)) = 1 - \epsilon$ 在连续分布的等式约定下成立；将其应用于 a^* 即得单次选择结论。对于序列结论，令 E_ℓ 为事件 $U(a_\ell^*) < BB_e(a_\ell^*)$ ，该事件基于选择 ℓ ，so $\Pr(E_\ell) = \epsilon$ 时校准的工作后验。由联合界 $\Pr(\bigcup_{\ell=1}^L E_\ell) \leq L\epsilon$ ，因此以至少 $1 - L\epsilon$ 的概率，没有被选候选低于其报告的下界。□

备注 4。 命题 4 是关于所选下界轨迹的校准陈述。它证明了将递增的最佳信念曲线解释为对更优最佳信念智能体的递增后验证据是合理的，前提是命题 1 中已陈述的工作后验假设成立。

分段固定准则有效性。设

$$0 = \tau_0 < \tau_1 < \dots < \tau_K \leq B$$

为实际转移时间。转移仅发生在某些槽位发生变化的预先指定检查点。设 $\mathcal{F}_{\tau_k}$ 为由档案、保留的效用记录、不可变审计日志、候选评估器、检查点统计量以及截至 τ_k 所做的所有随机选择生成的西格玛域。

命题 5 (分段固定准则有效性)。假设：(i) 在每个训练轮次内，依赖评估器的角色满足假设 2 中的平稳学习评估条件；(ii) 独立于评估器的角色使用固定的真实准则；(iii) 槽位转移仅发生在检查点边界；(iv) 每次转移后，擦除操作移除在新准则下无效的记录；(v) 所有导出的搜索统计量均从保留的记录中重新计算。那么，在 $\mathcal{F}_{\tau_k}$ 和 τ_k 处选择的准则向量的条件下，区间 $[\tau_k, \tau_{k+1})$ 是 a 固定准则二元结果搜索问题。特别地，对于该区间内每个符合条件的节点-角色-任务元组 (a, r, d) ，存在一个固定的训练轮次局部成功概率

$$p_{a,r,d}^{(k)} = \Pr(O = 1 \mid a, r, d, \kappa^{(k)}),$$

and the corresponding role-task balanced utility is fixed for the duration of the epoch.

证明。对训练轮次进行归纳。在 $\mathcal{F}_{\tau_k}$ 和 τ_k 处选择的准则向量的条件下，命题 2 给出在 $[\tau_k, \tau_{k+1})$ 开始时准则一致的档案：擦除操作移除在新准则下无效的每条记录，并且导出的统计量从保留的记录中重新计算，因此没有过期记录进入该训练轮次的统计量。区间内不再发生进一步转移，因此命题 3 适用，并且每个符合条件的 (a, r, d) 具有固定的训练轮次局部均值 $p^{(k)}_{a,r,d}$ ，其中角色-任务平衡效用是这些均值的固定加权平均。继承的树是 τ_k 处条件档案的一部分，并且不进入训练轮次内的结果过程，从而完成归纳。□

备注 5。该结果是训练轮次局部的：在条件化于检查点决策并应用选择性擦除后，每个已实现的训练轮次都与 HGM 搜索所使用的固定标准二元结果接口兼容（多训练轮次范围在备注 6 中界定）。

每个检查点处评估器锚点下界。上述备注是训练轮次局部的，因为任务智能体标准在每次转换时都会改变。评估器槽位是例外：其替换测试仅在固定锚点上评分（备注 3），因此在整个运行过程中，该槽位是在检查点处采样的单一固定标准选择，命题 4 适用于它。固定一个槽位，用 $U^{\text{gt}}(e)$

表示评估器的锚点效用（其在固定锚点上的准确率），用 $S_{e^{(k)}}^{\text{gt}}, 1 + L^{(k)} = BB_{\epsilon}^{\text{gt}}(e^{(k)}) = I_{\epsilon}^{-1}(1 + \frac{1}{e^{(k)}})$ 表示在其第 k 次转换时晋升的评估器的锚点最佳信念。

备注 6 (评估器锚点下界)。固定一个锚点标准在整个运行中保持不变的槽位，晋升的评估器为 $e^{(0)}, \dots, e^{(K)}$ ，锚点最佳信念值 $L^{(k)} = BB^{\text{gt}}$ 在每次晋升时测量。如果锚点工作后验是校准的，则以至少 $1 - K\epsilon$ 的概率，每个晋升的评估器的真实锚点效用联合满足

$U^{\text{gt}}(e^{(k)}) \geq L^{(k)}$ ；这是命题 4 的序列声明应用于固定锚点上的 K 个晋升评估器 ($L = K$)。这是每个检查点的下界陈述，结合备注 3 的最佳信念支配，并不保证已实现的锚点准确率单调提升。

指数级检查点开销。设 B 为搜索评估预算。固定检查点基数 $\rho > 1$ 和最小尺度 $h \geq 1$ 。考虑检查点机会

$$C_B(\rho, h) = \{ \lfloor h\rho^q \rfloor : q = 0, 1, \dots, Q \}, \text{ 其中 } Q \text{ 是满足 } h\rho^Q \leq B \text{ 的最大整数。}$$

命题 6 (指数检查点下的线性工作量)。若在时间 c 处的检查点最多可重新处理所有 c 个先前的槽依赖记录，则在 $C_B(\rho, h)$ 中所有检查点上重新处理的记录总数为 $O(B)$ ，常数至多为 $\frac{\rho}{\rho-1}$ 。

证明。在检查点 q 处，可重新处理的先前记录数至多为

$$\lfloor h\rho^q \rfloor \leq h\rho^q.$$

对所有检查点机会求和可得

$$h \sum_{q=0}^Q \rho^q = \frac{h(\rho^{Q+1} - 1)}{\rho - 1} \leq \frac{\rho B}{\rho - 1}.$$

对于固定的 $\rho > 1$ ，此为 $O(B)$ 。常见的基二调度是特例 $\rho = 2$ ，其界至多为 $2B$ 。□

注 7（密集均匀检查点）。 在每 h 步均匀检查点下，检查点 q 最多可重新处理 qh 个先前记录，因此在 B/h 个检查点上的总数为 $\sum_{q=1}^{\lfloor B/h \rfloor} qh = \Theta(B^2/h)$ ：对于固定的 h ，在预算上呈二次方而非线性。因此，在指数调度下，完全重放与线性暴露是兼容的。有界恢复仍是默认选择，因为它使效用转换可审计，并避免在缓存工件上花费不必要的评估。

G 局限性

由于计算约束，我们尚未在实验数量、超参数配置范围或持续时间方面探索完整的设计空间或 RQGM 引入的权衡。此外，我们的实证范围仍局限于特定领域：编码/多语言、论文写作/评审以及 IMO 评分/证明写作，每个领域均以 HGM-H 控制隔离。一个统一的、跨领域的、全角色树涵盖这些领域仍未测试。另外，我们所有的主要实验结果均仅依赖于 GPT-5.5（低）。

从形式化角度看，为 RQGM 提供的理论保证本质上是局部且受限的。工作后验仅具有均值一致性（命题 1），且我们的数学保证严格在训练轮次局部层面运作，既未约束转移次数、被擦除证据的累积遗憾，也未约束向全局最优智能体-评估器对的长期收敛（附录 F.4）。最佳信念曲线并非证明绝对收敛，而是表示校准的下界轨迹（命题 4），其中每次转移都会丢弃被擦除的信息，且继承的树拓扑可能反映为过时评估器优化的扩展选择。此外，由于超出我们控制的系统性波动（包括提供商侧模型更新、非确定性工具、硬件波动或未记录的提示修改），假设 2 在现实部署中无法严格成立。然而，这种波动是一个局限性，普遍适用于所有未完全控制整个软件和硬件栈的框架。

我们的评估度量和循环边界引入了进一步的不同约束。审稿人小组本质上衡量的是跨审稿人的接受行为，而非客观的、基于真实标准的科学价值，并且未对生成的论文或证明进行任何人工评分。由于我们的经验证据完全来自智力工件领域，这些动态可能无法推广到具有根本不同反馈结构的环境。

此外，演化评估层引入了锚点弱化的特定风险，即弱、噪声大或有偏的锚点允许评估器跨训练轮次漂移，特别是因为 APReS 决策和 IMO-GradingBench 评分本身并不完美。尽管这个问题只有在数学等完全可验证的领域中才能真正避免，但我们打算在未来版本中使我们的机制更少依赖良好的锚点数据集。

虽然我们的搜索循环是稳健的，但它仍然是手工设计的，这限制了递归自我改进的收益仅限于我们选定的基准以及由我们的任务智能体生成的输出（这些输出既受我们的基准影响，也受基础模型的预训练分布影响）。将可演化表面扩展到包括调度器和替换规则，将需要比本文分析的护栏机制显著更强大的护栏机制。